从 Hartree 到电子结构小项目：

面向量子化学与 Python 零基础的逐步复现教程

simple_hf_water 项目教学讲义

2026 年 4 月 13 日

目录

第一部分　第一部分：先把 HF 骨架真正学明白 **7**

第一章　这一次我们到底要讲到哪里 **9**

第二章　零基础读者先要建立的三件事 **11**

2.1　什么叫“量子化学程序” 11

2.2　什么叫“用代码复现公式” 11

2.3　什么叫“读懂代码” 12

第三章　先补 Python 语法：这份项目到底用到了哪些基础知识 **13**

3.1　为什么这一章必须先讲 13

3.2　第一眼先看什么：一段 Python 代码的外形 13

3.3　缩进为什么这么重要 14

3.4　`import` 到底在干什么 14

3.5　`@dataclass` 是什么 14

3.6　类型标注要怎么看 15

3.7　赋值、`None` 和整数除法 15

3.8　条件判断：`if / elif / else` 15

3.9　循环：`for` 和 `range` 16

3.10　列表、元组、字典在项目里的角色 16

3.11　列表推导式是什么 16

3.12　切片是什么 17

3.13　矩阵乘法和 NumPy 17

3.14　`np.einsum` 不要怕 17

3.15　异常处理：`raise` 和 `try/except` 17

3.16　读后面源码时的一个固定顺序 18

第四章　程序的总路线：从命令行到求解器 **19**

4.1　最外层入口脚本只有 7 行 19

4.2 命令行是如何被组织起来的 19
4.3 输入几何是如何被整理成统一对象的 23
4.4 主函数里 RHF/UHF 的分发路线 24

第五章 先把“分子输入层”讲透：geometry.py 29
5.1 为什么一定要有 geometry.py 29
5.2 最前面的常数和数据类 29
5.3 输入规范化与单位换算 31
5.4 字符串几何如何变成数值数组 32
5.5 这一册真正需要的几何函数其实不多 34
5.6 读取 XYZ 文件与默认水分子 35

第六章 RHF 理论：从 Hartree 走向真正可运行的自洽场 37
6.1 为什么 Hartree 不够 37
6.2 RHF 的矩阵方程 37

第七章 把 RHF 源码逐段讲透：rhf.py 39
7.1 结果对象为什么先定义成 dataclass 39
7.2 DIIS 缓冲器是怎么设计的 40
7.3 把 MoleculeSpec 变成 PySCF 分子对象 41
7.4 为什么还要有 build_rhf_reference_mf 42
7.5 正交化、密度矩阵和 Fock 矩阵 43
7.6 为什么 RHF 必须先检查闭壳层条件 44
7.7 最关键的函数：run_rhf 45

第八章 RHF 跑通之后，为什么还要学 UHF 49
8.1 闭壳层体系和开壳层体系 49
8.2 UHF 的基本思想 49

第九章 把 UHF 源码逐段讲透：uhf.py 51
9.1 结果对象比 RHF 多了什么 51
9.2 单自旋密度矩阵 52
9.3 UHF 的 Fock 矩阵是如何构造的 52
9.4 为什么要把 alpha 和 beta 拼成块矩阵 53
9.5 输入检查和电子数分配 53
9.6 $\langle S^2 \rangle$ 是怎么计算的 54

9.7 最关键的函数：run_uhf 55
9.8 为什么还有 build_uhf_reference_mf 59

第十章 把“从输入到输出”的整条路径串起来 61
10.1 RHF 模式下，程序一步一步经历了什么 61
10.2 UHF 模式下，多出来了什么 61

第十一章 如果你要自己从零重写这一部分，应该按什么顺序 63
11.1 最推荐的复现顺序 63
11.2 每完成一步，你应该检查什么 63

第二部分 第二部分：从相关能到结构分析 65

第十二章 MP2 与 UMP2：第一次真正看到“电子相关” 67
12.1 为什么 RHF/UHF 之后先学 MP2 67
12.2 先把 `mp2.py` 整体看一遍 67
12.3 `transform_ao_eri_to_ovov()` 到底在做什么 69
12.4 `run_mp2()` 应该怎样读 70
12.5 UMP2：为什么会比 MP2 多出三个相关能分量 71

第十三章 DFT：RKS、UKS 与 `--xc` 泛函选择 77
13.1 为什么 DFT 看起来像 HF，却又不是 HF 77
13.2 先看 `rks.py`：闭壳层 DFT 文件其实很整洁 77
13.3 `--xc` 到底怎么理解 80
13.4 回调函数 `callback` 为什么值得你学习 80
13.5 再看 `uks.py`：它和 UHF 的关系很直接 81

第十四章 CCSD：把单参考相关方法再推进一步 85
14.1 CCSD 和 MP2 的关系 85
14.2 完整看一遍 `ccsd.py` 85

第十五章 几何优化与扫描：从单点能走向结构探索 89
15.1 几何优化本质上在做什么 89
15.2 先看 `optimize.py` 的文件地图 90
15.3 `optimize_geometry()` 的主循环该怎么读 93
15.4 扫描：为什么要同时支持 rigid 和 relaxed 97

15.5 `scan.py` 文件虽然长，但可以分成四层 98
15.6 读 rigid scan 和 relaxed scan 的主循环 101

第十六章 频率分析：检查结构是极小值还是鞍点 111
16.1 频率分析为什么离不开 Hessian 111
16.2 完整看一遍 `frequency.py` 112

第十七章 学完这一轮之后，下一步该怎么继续 117
17.1 最推荐的继续路线 117
17.2 哪些资料最适合从“会跑”走向“会写” 117

第十八章 本册的最后总结 119

附录：本册建议实际运行的命令 121

第一部分

第一部分：先把 HF 骨架真正学明白

第一章　这一次我们到底要讲到哪里

这份讲义现在分成两大部分。第一部分还是严格按零基础节奏来，从 Hartree 一路走到 RHF、DIIS、UHF；第二部分再在这个骨架上继续往前，补上相关能、DFT、结构优化、扫描和频率分析。

整个项目的主线现在是：

$$\begin{aligned}\text{Hartree} &\longrightarrow \text{Hartree–Fock} \longrightarrow \text{RHF/UHF} \longrightarrow \text{MP2/UMP2}\\ &\longrightarrow \text{RKS/UKS} \longrightarrow \text{CCSD} \longrightarrow \text{优化/扫描/频率.}\end{aligned} \tag{1.1}$$

阅读路线

如果你是第一次接触量子化学编程，请一定先把前半部分学扎实，再进入后半部分。因为 MP2、DFT、CCSD、优化这些模块虽然名字不同，但几乎都建立在同一套基础对象上：分子对象、积分、轨道系数、密度矩阵、Fock/KS 矩阵、收敛控制和 PySCF 接口。

这份教程真正想做到的是：

1. 让你知道 Hartree 和 Hartree–Fock 到底在解决什么问题。
2. 让你把 RHF 的每一个数学对象，和 Python 里的变量一一对应起来。
3. 让你把 `simple_hf/geometry.py`、`simple_hf/rhf.py`、`simple_hf/uhf.py` 这些文件读懂。
4. 让你明白程序真正的执行路线：命令行参数是如何一路走到 RHF/UHF、MP2、DFT、优化和频率模块里的。
5. 让你看到“一个小型量子化学程序”是怎样从单点能，逐步成长到结构分析工具箱的。

它不是“看完就会背公式”，而是希望你看完后能自己重新敲出一个简化版电子结构程序，并且知道每多加一个功能，代码结构为什么会那样长出来。

第二章　零基础读者先要建立的三件事

2.1　什么叫“量子化学程序”

最朴素地说，一个量子化学程序做的是：

1. 接收分子结构；
2. 选定一种电子结构方法和基组；
3. 计算电子能量；
4. 在需要时，再进一步计算轨道、梯度、频率等量。

在我们的项目里，哪怕最后功能很多，这个骨架从来没有变：

$$\text{输入分子} + \text{输入方法} \longrightarrow \text{构造矩阵} \longrightarrow \text{迭代求解} \longrightarrow \text{输出结果}. \tag{2.1}$$

2.2　什么叫“用代码复现公式”

初学者最容易卡住的地方，是明明公式看懂了，却不知道怎么落成代码。这里有一个非常重要的观念：

> 量子化学程序并不是“把公式原样打进电脑”，而是把公式变成一组矩阵、数组、循环和收敛判据。

例如，RHF 中最核心的公式

$$\mathbf{FC} = \mathbf{SC}\varepsilon \tag{2.2}$$

在程序里不会以“公式”的样子出现，而会变成：

- 用 `overlap` 表示 $\mathbf{S}$；
- 用 `fock` 表示 $\mathbf{F}$；
- 用 `coeff` 表示 $\mathbf{C}$；
- 用 `orbital_energies` 表示 ε 的对角元。

2.3　什么叫“读懂代码”

对于这一类项目，真正的“读懂”不是指你能从头背出每一行，而是你回答得出下面这些问题：

- 这一行代码对应哪一个数学量？
- 这个函数的输入和输出分别是什么物理对象？
- 这一段循环为什么必须反复执行？
- 这里为什么需要误差矩阵、收敛阈值、DIIS 缓存？

如果你能回答这些问题，你就已经不是在“抄代码”，而是在真正学习量子化学编程。

第三章　先补 Python 语法：这份项目到底用到了哪些基础知识

3.1　为什么这一章必须先讲

如果这份讲义真的是给零基础读者用，那么在讲量子化学程序之前，必须先解决一个更前面的问题：

> 你是否读得懂 Python 代码本身？

很多时候初学者不是败在量子化学公式，而是败在这些最基础的地方：

- 不知道 `def` 是什么意思；
- 不知道 `return` 为什么重要；
- 看不懂 `list[str]`、`tuple[int, int]` 这种类型标注；
- 不知道 `[:, :nocc]` 为什么能切片；
- 看见 `@dataclass`、`np.einsum()`、`strict=True` 就开始紧张。

所以这一章的目标很明确：

> 不是系统讲完整门 Python，而是把读懂本项目源码所必需的语法一次讲清楚。

3.2　第一眼先看什么：一段 Python 代码的外形

来看一个很小的例子：

```python
def build_density(coefficients: np.ndarray, nocc: int) -> np.ndarray:
    occupied = coefficients[:, :nocc]
    return 2.0 * occupied @ occupied.T
```

如果你完全是 Python 小白，请先只认出四个层次：

1. `def` 开头，说明这是“定义一个函数”。

2. 括号里是输入参数。
3. 冒号后面缩进的部分，是函数真正执行的内容。
4. `return` 表示“把结果返回出去”。

3.3 缩进为什么这么重要

Python 不是靠大括号 `{}` 区分代码块，而是靠缩进。

```
if use_diis:
    diis_error = compute_diis_error(fock, density, s, x)
    diis_helper.push(fock, diis_error)
    fock_to_diagonalize = diis_helper.extrapolate()
else:
    fock_to_diagonalize = fock
```

这表示：

- 条件为真时，执行第一块缩进代码；
- 条件为假时，执行 `else` 那一块。

3.4 import 到底在干什么

项目里一开头经常有：

```
from dataclasses import dataclass
import numpy as np
from pyscf import gto, scf
```

它们分别表示：

- 从 `dataclasses` 里导入 `dataclass`
- 导入 `numpy` 并起别名 `np`
- 从 `pyscf` 中取出 `gto` 和 `scf`

3.5 @dataclass 是什么

项目里经常有：

```
@dataclass
class RHFResult:
    energy: float
    electronic_energy: float
    nuclear_repulsion: float
```

你可以先把它理解成：

一个专门拿来装数据的类。

它的作用不是让程序“更神秘”，而是让结果对象更整齐、更适合后续复用。

3.6 类型标注要怎么看

例如：

```
def parse_atom_string(atom: str) -> tuple[list[str], np.ndarray]:
```

你可以把它拆成：

- `atom: str`：输入预计是字符串；
- 箭头 `->` 后面：返回值类型；
- `tuple[list[str], np.ndarray]`：返回一个二元组。

类型标注最主要是帮助人读代码，不是增加额外负担。

3.7 赋值、None 和整数除法

```
h_core = t + v
nocc = mol.nelectron // 2
previous_energy = None
```

这里：

- `=` 表示把右边结果交给左边变量名；
- `//` 表示整数除法；
- `None` 表示“现在还没有一个真正的值”。

3.8 条件判断：if / elif / else

```
if method == "rhf":
    ...
elif method == "uhf":
    ...
else:
    raise ValueError("Unsupported method")
```

这就是程序分流的基本方式。

3.9 循环：for 和 range

```
for iteration in range(1, max_iter + 1):
    ...
```

这可以读成：

> 让 iteration 从 1 走到 max_iter，每次都执行一次循环体。

3.10 列表、元组、字典在项目里的角色

列表 list

```
history: list[float] = []
```

列表适合装“会不断增长的一串东西”，比如 SCF 每一步的能量历史。

元组 tuple

```
def parse_scan_atoms(text: str) -> tuple[int, ...]:
```

元组更适合表示“固定位置的一组值”，比如键长、键角、二面角的原子编号。

字典 dict

```
SUPPORTED_BASIS_ALIASES = {
    "sto-3g": "sto-3g",
    "6-31g*": "6-31g(d)",
}
```

字典就是键值映射表。

3.11 列表推导式是什么

项目里有：

```
cleaned = [line.strip() for line in lines if line.strip()]
```

你可以把它先翻译成普通话：

> 把每一行去掉空白；如果去掉之后不是空行，就保留下来。

3.12 切片是什么

项目中非常常见：

```
occupied = coefficients[:, :nocc]
mo_occ[:nocc] = 2.0
```

这里：

- : 表示“这一维全取”；
- :nocc 表示“从开头取到第 nocc 个之前”。

3.13 矩阵乘法和 NumPy

```
return 2.0 * occupied @ occupied.T
```

@ 表示矩阵乘法。.T 表示转置。np.array、np.linalg.eigh、np.linalg.norm 都说明我们在用 NumPy 做线性代数。

3.14 np.einsum 不要怕

例如：

```
coulomb = np.einsum("ls,mnls->mn", density, eri, optimize=True)
```

你可以先把它理解成：

> 一种把多指标求和压缩写成一行的工具。

现在先知道它在做“积分张量缩并”就够了，后面随着经验增长再回头专门理解它。

3.15 异常处理：raise 和 try/except

```
raise ValueError("Unsupported basis")
```

表示程序要明确报错，而不是静悄悄继续算。

```
try:
    coefficients = np.linalg.solve(b_matrix, rhs)[:-1]
except np.linalg.LinAlgError:
    return self.fock_matrices[-1]
```

表示先尝试执行，如果失败，就走备用方案。

3.16 读后面源码时的一个固定顺序

从这一章开始，建议你以后每次看到一段新代码，都按这个顺序读：

1. 这是一段定义，还是一段执行？
2. 输入参数是什么？
3. 返回值是什么？
4. 缩进层级怎么分块？
5. 变量名分别对应什么物理量？

第四章　程序的总路线：从命令行到求解器

4.1　最外层入口脚本只有 7 行

先看整个项目最外层的入口：

```python
#!/usr/bin/env python3

from simple_hf.cli import main


if __name__ == "__main__":
    main()
```

这个文件非常短，但它的作用非常重要。它告诉我们：

- 真正的主逻辑不在这个脚本里；
- 这个脚本只负责启动程序；
- 所有实际功能都在 `simple_hf.cli` 的 `main()` 中。

这是一种很好的结构设计。因为以后无论你给项目增加多少方法，入口都不需要改动。

4.2　命令行是如何被组织起来的

接下来要看 `simple_hf/cli.py` 中最前面的参数解析部分：

```python
def build_parser() -> argparse.ArgumentParser:
    parser = argparse.ArgumentParser(
        description="Minimal teaching example for HF/DFT/post-HF methods, geometry
            optimization, and internal-coordinate scans."
    )
    input_group = parser.add_mutually_exclusive_group()
    input_group.add_argument(
        "--xyz",
        type=str,
        help="Path to an XYZ geometry file.",
    )
```

```python
    input_group.add_argument(
        "--geometry",
        type=str,
        help="Inline geometry such as 'O 0 0 0; H 0 -0.757 0.586; H 0 0.757 0.586'.",
    )

    parser.add_argument(
        "--basis",
        type=str,
        default="sto-3g",
        help="Basis set name. Supported: sto-3g, 6-31G(d), 6-31G*.",
    )
    parser.add_argument(
        "--method",
        type=str,
        default="rhf",
        choices=["rhf", "uhf", "rks", "uks", "mp2", "ump2", "ccsd"],
        help="Electronic structure method to run.",
    )
    parser.add_argument(
        "--xc",
        type=str,
        default="b3lyp",
        help="DFT exchange-correlation functional for RKS/UKS, such as 'lda,vwn', 'pbe', or 'b3lyp'.",
    )
    parser.add_argument("--charge", type=int, default=0, help="Total molecular charge.")
    parser.add_argument(
        "--spin",
        type=int,
        default=0,
        help="2S = N(alpha) - N(beta). RHF/RKS/MP2/CCSD require spin = 0; UHF/UKS support open-shell cases.",
    )
    parser.add_argument(
        "--unit",
        type=str,
        default="Angstrom",
        choices=["Angstrom", "Bohr"],
        help="Coordinate unit for --geometry input. XYZ input is assumed to use Angstrom.",
    )
    parser.add_argument("--max-iter", type=int, default=100, help="Maximum SCF iterations.")
    parser.add_argument("--energy-tol", type=float, default=1.0e-10, help="Energy convergence threshold.")
    parser.add_argument("--density-tol", type=float, default=1.0e-8, help="Density
```

```
        convergence threshold.")
    parser.add_argument(
        "--optimize",
        action="store_true",
        help="Run a geometry optimization. Supported for RHF, UHF, RKS, and UKS.",
    )
    parser.add_argument(
        "--frequency",
        action="store_true",
        help="Run a harmonic frequency analysis. Supported for RHF, UHF, RKS, and UKS.",
    )
    parser.add_argument(
        "--scan",
        type=str,
        choices=["rigid", "relaxed"],
        help="Run an internal-coordinate scan. 'rigid' keeps other coordinates fixed; '
            relaxed' optimizes all others.",
    )
    parser.add_argument(
        "--scan-coordinate",
        type=str,
        default="angle",
        choices=["bond", "angle", "dihedral"],
        help="Internal coordinate type for scanning.",
    )
    parser.add_argument(
        "--scan-atoms",
        type=str,
        help="1-based atom indices for the scan coordinate. bond: '1,2'; angle: '2,1,3';
            dihedral: '1,2,3,4'.",
    )
    parser.add_argument("--scan-start", type=float, help="Scan start value. Degrees for
        angle/dihedral, geometry unit for bond.")
    parser.add_argument("--scan-stop", type=float, help="Scan stop value. Degrees for
        angle/dihedral, geometry unit for bond.")
    parser.add_argument("--scan-points", type=int, default=7, help="Number of scan points.
        ")
    parser.add_argument(
        "--scan-output",
        type=str,
        help="Optional CSV file path for scan results.",
    )
    parser.add_argument(
        "--constraint-k",
        type=float,
        default=50.0,
```

```
        help="Harmonic penalty strength for relaxed angle scans in Eh/rad^2.",
    )
    parser.add_argument(
        "--opt-max-steps",
        type=int,
        default=30,
        help="Maximum geometry optimization steps.",
    )
    parser.add_argument(
        "--grad-tol",
        type=float,
        default=1.0e-4,
        help="Maximum Cartesian gradient threshold in Eh/Bohr.",
    )
    parser.add_argument(
        "--opt-energy-tol",
        type=float,
        default=1.0e-8,
        help="Optimization energy convergence threshold in Eh.",
    )
    parser.add_argument(
        "--max-step-size",
        type=float,
        default=0.2,
        help="Maximum geometry step size in Bohr.",
    )
    parser.add_argument(
        "--no-diis",
        action="store_true",
        help="Disable DIIS and use plain SCF iteration.",
    )
    parser.add_argument(
        "--diis-space",
        type=int,
        default=6,
        help="Number of Fock/error matrices kept in the DIIS subspace.",
    )
    parser.add_argument(
        "--show-history",
        action="store_true",
        help="Print the SCF total energy at each iteration.",
    )
    return parser
```

请先不要害怕代码长。对初学者来说，这一大段其实只在做两件事：

1. 定义用户可以输入哪些参数；
2. 给这些参数设置默认值和帮助说明。

对我们这一册最重要的参数有：

- `--xyz`：从 XYZ 文件读分子；
- `--geometry`：直接写内联几何；
- `--basis`：选择基组；
- `--method`：选择 `rhf` 或 `uhf`；
- `--charge`：分子电荷；
- `--spin`：自旋多重度相关输入，这里用的是 $2S = N_\alpha - N_\beta$；
- `--max-iter`、`--energy-tol`、`--density-tol`：SCF 收敛控制；
- `--no-diis`、`--diis-space`：是否开启 DIIS。

虽然这个 `parser` 里已经有很多“后续章节才会用到”的选项，比如 `--scan`、`--frequency` 等，但你现在只需要把它看作：

程序和用户沟通的总开关面板。

4.3 输入几何是如何被整理成统一对象的

继续看 `cli.py` 中真正把参数转成“分子描述”的部分：

```python
def parse_scan_atoms(text: str) -> tuple[int, ...]:
    try:
        values = [int(token.strip()) for token in text.split(",")]
    except ValueError as exc:
        raise ValueError("--scan-atoms must look like '1,2' or '2,1,3' or '1,2,3,4'.") from exc
    if min(values) < 1:
        raise ValueError("--scan-atoms uses 1-based indices, so all values must be >= 1.")
    return tuple(value - 1 for value in values)


def build_spec_from_args(args: argparse.Namespace) -> MoleculeSpec:
    basis = normalize_basis_name(args.basis)

    if args.xyz:
        atom, title = read_xyz_geometry(args.xyz)
        return MoleculeSpec(
```

```
            atom=atom,
            basis=basis,
            charge=args.charge,
            spin=args.spin,
            unit="Angstrom",
            title=title or Path(args.xyz).stem,
        )

    if args.geometry:
        return MoleculeSpec(
            atom=parse_inline_geometry(args.geometry),
            basis=basis,
            charge=args.charge,
            spin=args.spin,
            unit=args.unit,
            title="Inline geometry",
        )

    default = default_water_spec(basis=basis)
    return MoleculeSpec(
        atom=default.atom,
        basis=basis,
        charge=args.charge,
        spin=args.spin,
        unit=default.unit,
        title=default.title,
    )
```

这一段特别值得初学者学，因为它体现了一个很常见的工程思想：

> 无论用户是从文件输入、命令行输入，还是完全不输入，都要尽快变成同一种内部数据结构。

这里那个统一的数据结构就是 `MoleculeSpec`。有了它之后，后面的代码就不用关心“用户到底是怎么输入几何的”。

4.4 主函数里 RHF/UHF 的分发路线

下面是和本册最相关的主流程分发部分：

```
def main() -> None:
    parser = build_parser()
    args = parser.parse_args()
    if args.diis_space < 2:
        parser.error("--diis-space must be at least 2.")
    if args.scan_points is not None and args.scan_points < 2:
```

```
        parser.error("--scan-points must be at least 2.")
    if args.optimize and args.scan:
        parser.error("--optimize and --scan cannot be used together.")
    if args.frequency and args.scan:
        parser.error("--frequency and --scan cannot be used together.")
    if args.optimize and args.method not in {"rhf", "uhf", "rks", "uks"}:
        parser.error("--optimize currently supports --method rhf, uhf, rks, or uks.")
    if args.frequency and args.method not in {"rhf", "uhf", "rks", "uks"}:
        parser.error("--frequency currently supports --method rhf, uhf, rks, or uks.")
    spec = build_spec_from_args(args)
    if args.scan:
        if args.scan_atoms is None or args.scan_start is None or args.scan_stop is None:
            parser.error("--scan requires --scan-atoms, --scan-start, and --scan-stop.")
        scan_atoms = parse_scan_atoms(args.scan_atoms)
        expected_arity = coordinate_arity(args.scan_coordinate)
        if len(scan_atoms) != expected_arity:
            parser.error(f"--scan-coordinate {args.scan_coordinate} requires exactly {
                expected_arity} atom indices.")
        natom = len(spec.atom.splitlines())
        if max(scan_atoms) >= natom:
            parser.error(f"--scan-atoms refers to atom index > {natom}.")
        if args.scan == "rigid":
            scan_result = rigid_scan(
                spec,
                method=args.method,
                xc=args.xc,
                coordinate_type=args.scan_coordinate,
                atoms=scan_atoms,
                start_value=args.scan_start,
                stop_value=args.scan_stop,
                num_points=args.scan_points,
                max_iter=args.max_iter,
                e_tol=args.energy_tol,
                d_tol=args.density_tol,
                use_diis=not args.no_diis,
                diis_space=args.diis_space,
            )
        else:
            if args.method not in {"rhf", "uhf", "rks", "uks"}:
                parser.error("--scan relaxed currently supports --method rhf, uhf, rks, or
                    uks.")
            scan_result = relaxed_scan(
                spec,
                method=args.method,
                xc=args.xc,
                coordinate_type=args.scan_coordinate,
```

```
            atoms=scan_atoms,
            start_value=args.scan_start,
            stop_value=args.scan_stop,
            num_points=args.scan_points,
            penalty_k=args.constraint_k,
            max_opt_steps=args.opt_max_steps,
            grad_tol=args.grad_tol,
            opt_energy_tol=args.opt_energy_tol,
            max_step_size=args.max_step_size,
            max_iter=args.max_iter,
            scf_e_tol=args.energy_tol,
            scf_d_tol=args.density_tol,
            use_diis=not args.no_diis,
            diis_space=args.diis_space,
        )
    if args.scan_output:
        write_scan_csv(scan_result, args.scan_output)
    print_scan_result(scan_result)
    return

if args.optimize:
    opt_result = optimize_geometry(
        spec,
        method=args.method,
        xc=args.xc,
        max_opt_steps=args.opt_max_steps,
        grad_tol=args.grad_tol,
        energy_tol=args.opt_energy_tol,
        max_step_size=args.max_step_size,
        max_iter=args.max_iter,
        scf_e_tol=args.energy_tol,
        scf_d_tol=args.density_tol,
        use_diis=not args.no_diis,
        diis_space=args.diis_space,
    )
    print_optimization_result(spec, opt_result, args.show_history)
    if args.frequency:
        print()
        freq_result = run_frequency(
            opt_result.optimized_spec,
            method=args.method,
            xc=args.xc,
            max_iter=args.max_iter,
            e_tol=args.energy_tol,
            d_tol=args.density_tol,
            use_diis=not args.no_diis,
```

```
            diis_space=args.diis_space,
        )
        print_frequency_result(opt_result.optimized_spec, freq_result)
    return

if args.frequency:
    freq_result = run_frequency(
        spec,
        method=args.method,
        xc=args.xc,
        max_iter=args.max_iter,
        e_tol=args.energy_tol,
        d_tol=args.density_tol,
        use_diis=not args.no_diis,
        diis_space=args.diis_space,
    )
    print_frequency_result(spec, freq_result)
    return

mol = build_molecule(spec)
rhf_result = run_rhf(
    mol,
    max_iter=args.max_iter,
    e_tol=args.energy_tol,
    d_tol=args.density_tol,
    use_diis=not args.no_diis,
    diis_space=args.diis_space,
) if args.method in {"rhf", "mp2", "ccsd"} else None
if args.method == "rhf":
    assert rhf_result is not None
    print_rhf_result(spec, rhf_result, mol.nao_nr(), mol.nelectron, args.show_history)
    return

if args.method == "rks":
    rks_result = run_rks(
        mol,
        xc=args.xc,
        max_iter=args.max_iter,
        e_tol=args.energy_tol,
        d_tol=args.density_tol,
        use_diis=not args.no_diis,
        diis_space=args.diis_space,
    )
    print_rks_result(spec, rks_result, mol.nao_nr(), mol.nelectron, args.show_history)
    return

```

```
    if args.method == "mp2":
        assert rhf_result is not None
        mp2_result = run_mp2(mol, rhf_result)
        print_mp2_result(spec, rhf_result, mp2_result, mol.nao_nr(), mol.nelectron, args.
            show_history)
        return

    if args.method == "ccsd":
        assert rhf_result is not None
        ccsd_result = run_ccsd(mol, rhf_result)
        print_ccsd_result(spec, rhf_result, ccsd_result, mol.nao_nr(), mol.nelectron, args.
            show_history)
        return
```

请你重点抓住这一段代码里的执行顺序：

1. 建立 parser；
2. 读取 args；
3. 调用 build_spec_from_args(args) 变成 spec；
4. 调用 build_molecule(spec) 变成 PySCF 的 mol；
5. 根据 args.method 决定是跑 run_rhf 还是 run_uhf；
6. 最后调用打印函数输出结果。

你可以把整个项目最核心的调用链记成一句话：

$$\texttt{main()} \longrightarrow \texttt{build_spec_from_args()} \longrightarrow \texttt{build_molecule()} \longrightarrow \texttt{run_rhf()/run_uhf()}. \tag{4.1}$$

只要这个调用链在脑子里清楚，后面读代码就不会乱。

第五章　先把“分子输入层”讲透：geometry.py

5.1　为什么一定要有 geometry.py

很多初学者会以为量子化学程序的重点全在 Fock 矩阵。其实不是。一个程序要想稳定运行，首先必须把输入整理干净。

我们的 `geometry.py` 就是在负责这些事情：

- 单位换算；
- 基组名称规范化；
- 内联几何的解析；
- XYZ 文件读取；
- 默认水分子构型；
- 一些后续结构功能会用到的几何工具。

5.2　最前面的常数和数据类

```python
from __future__ import annotations

from dataclasses import dataclass
from pathlib import Path

import numpy as np


BOHR_TO_ANGSTROM = 0.529177210903
ANGSTROM_TO_BOHR = 1.0 / BOHR_TO_ANGSTROM


@dataclass(frozen=True)
class MoleculeSpec:
    atom: str
    basis: str = "sto-3g"
    charge: int = 0
```

```
    spin: int = 0
    unit: str = "Angstrom"
    title: str = "Molecule"


SUPPORTED_BASIS_ALIASES = {
    "sto-3g": "sto-3g",
    "6-31g(d)": "6-31g(d)",
    "6-31g*": "6-31g(d)",
}
```

这一段要讲清楚四个概念。

1. 为什么要有 BOHR_TO_ANGSTROM

量子化学里最常见的长度单位有两个：

- Bohr
- Angstrom

程序内部很多量在原子单位下更自然，而用户输入几何时通常更习惯 Angstrom。所以单位转换常数是基础设施，不是可有可无的小装饰。

2. MoleculeSpec 到底是什么

MoleculeSpec 是一个“分子描述对象”。它不是 PySCF 的分子对象，也不是电子结构结果，而只是把下面这些输入绑在一起：

- 原子坐标字符串 atom
- 基组 basis
- 电荷 charge
- 自旋 spin
- 单位 unit
- 标题 title

你可以把它理解成“分子在进入求解器之前的统一包装盒”。

3. 为什么要有基组别名表

SUPPORTED_BASIS_ALIASES 看起来很简单，但它体现的是“程序输入要容错”的思想。例如：

- 用户可能写 6-31G*

- 程序内部只想统一成 `6-31g(d)`

于是就需要别名映射。

4. 这里顺手复习一下 Python 语法

这一小段代码里其实已经包含了很多项目会反复出现的基础语法：

- `@dataclass`：声明数据类；
- `class MoleculeSpec:`：定义一个类；
- `atom: str`：字段的类型标注；
- 花括号 `{}`：定义字典；
- `"6-31g*": "6-31g(d)"`：字典里的键值对。

所以你在读源码时，最好同时问自己两个问题：

1. 这段代码在物理上想表达什么？
2. 这段代码在 Python 语法上属于什么结构？

5.3 输入规范化与单位换算

```python
def normalize_basis_name(name: str) -> str:
    normalized = name.strip().lower()
    if normalized not in SUPPORTED_BASIS_ALIASES:
        supported = ", ".join(sorted(SUPPORTED_BASIS_ALIASES))
        raise ValueError(
            f"Unsupported basis '{name}'. Supported options are: {supported}."
        )
    return SUPPORTED_BASIS_ALIASES[normalized]


def convert_coords(coords: np.ndarray, from_unit: str, to_unit: str) -> np.ndarray:
    if from_unit == to_unit:
        return np.array(coords, copy=True)
    if from_unit == "Angstrom" and to_unit == "Bohr":
        return np.array(coords, copy=True) * ANGSTROM_TO_BOHR
    if from_unit == "Bohr" and to_unit == "Angstrom":
        return np.array(coords, copy=True) * BOHR_TO_ANGSTROM
    raise ValueError(f"Unsupported unit conversion from {from_unit} to {to_unit}.")


def convert_length_value(value: float, from_unit: str, to_unit: str) -> float:
```

```
    coords = np.array([[value, 0.0, 0.0]])
    converted = convert_coords(coords, from_unit, to_unit)
    return float(converted[0, 0])
```

这几段函数虽然不复杂，但非常值得逐行读。

normalize_basis_name

它做的是：

1. 去掉首尾空格；
2. 转成小写；
3. 检查是不是允许的基组名字；
4. 如果是别名，就映射到规范写法。

这就是非常标准的“输入清洗”。

convert_coords

这一段反映的是很典型的工程思维：

- 如果输入和输出单位一样，直接返回拷贝；
- 如果是 Angstrom 到 Bohr，就乘转换因子；
- 如果是 Bohr 到 Angstrom，也一样；
- 其余情况一律报错。

这种写法的优点是：规则写死，行为清楚，不会悄悄产生模糊结果。

convert_length_value

这个函数很有意思。它不是直接写一条“标量换算公式”，而是先把标量包装成一个一维坐标，再复用 `convert_coords`。这种写法的好处是：

尽量不要重复维护两套转换逻辑。

5.4 字符串几何如何变成数值数组

```
def _normalize_geometry_lines(lines: list[str]) -> str:
    cleaned = [line.strip() for line in lines if line.strip()]
    if not cleaned:
```

```
        raise ValueError("No atomic coordinates were provided.")

    for line in cleaned:
        tokens = line.split()
        if len(tokens) < 4:
            raise ValueError(
                f"Invalid geometry line: '{line}'. Expected format like 'O 0.0 0.0 0.0'."
            )
    return "\n".join(cleaned)


def parse_inline_geometry(text: str) -> str:
    raw_lines = []
    for chunk in text.replace(";", "\n").splitlines():
        raw_lines.append(chunk)
    return _normalize_geometry_lines(raw_lines)


def parse_atom_string(atom: str) -> tuple[list[str], np.ndarray]:
    lines = _normalize_geometry_lines(atom.splitlines()).splitlines()
    symbols: list[str] = []
    coords = []
    for line in lines:
        symbol, x, y, z = line.split()[:4]
        symbols.append(symbol)
        coords.append([float(x), float(y), float(z)])
    return symbols, np.array(coords, dtype=float)


def format_atom_string(symbols: list[str], coords: np.ndarray) -> str:
    lines = [
        f"{symbol:<2s} {xyz[0]: .10f} {xyz[1]: .10f} {xyz[2]: .10f}"
        for symbol, xyz in zip(symbols, coords, strict=True)
    ]
    return "\n".join(lines)


def unit_vector(vector: np.ndarray) -> np.ndarray:
    norm = float(np.linalg.norm(vector))
    if norm < 1.0e-14:
        raise ValueError("Cannot normalize a near-zero vector.")
    return vector / norm
```

这一段是初学者一定要读懂的“字符串处理 + 数值化”代码。

_normalize_geometry_lines

这个函数先做两件事：

- 去掉空行和多余空白；
- 检查每一行至少有“元素符号 + 三个坐标”。

也就是说，它在做最基本的输入合法性检查。

parse_inline_geometry

用户可能输入：

```
--geometry "O 0 0 0; H 0 -0.757 0.586; H 0 0.757 0.586"
```

程序要把里面的分号变成换行，再交给统一的规范化函数处理。

parse_atom_string

这个函数特别重要。因为从这一刻开始，原本的文本几何被拆成了两个真正可计算的对象：

- `symbols`：元素符号列表
- `coords`：形状为 `(natom, 3)` 的 NumPy 数组

这一步就是从“文本输入”转向“数值计算”的分界线。

如果你是 Python 小白，这里还应该顺手注意两个语法点：

- `for line in lines`：是逐个遍历一组数据；
- `return symbols, np.array(...)` 表示函数一次返回两个对象，外面可以用两个变量分别接收。

format_atom_string

它做的正好是反操作：把元素和坐标重新拼成字符串。这对于后面优化、扫描时输出新几何非常有用。

5.5 这一册真正需要的几何函数其实不多

你会发现 `geometry.py` 后面还有很多函数，比如：

- `angle_radians`
- `bond_length`

- set_angle_rigid
- set_bond_length_rigid
- set_dihedral_rigid

这些函数主要是为后面的扫描与优化服务。因为这一册只讲到 UHF，所以你现在不用强迫自己把它们全吃透。

但有一件事你可以先知道：

> geometry.py 不是“只管读输入”的文件，它后来还逐渐成长成了“结构工具箱”。

5.6 读取 XYZ 文件与默认水分子

```python
def read_xyz_geometry(path: str) -> tuple[str, str]:
    xyz_path = Path(path)
    content = xyz_path.read_text(encoding="utf-8").splitlines()
    if len(content) < 3:
        raise ValueError("XYZ file is too short.")

    try:
        natom = int(content[0].strip())
    except ValueError as exc:
        raise ValueError("First line of XYZ file must be the atom count.") from exc

    title = content[1].strip() or xyz_path.stem
    coord_lines = content[2 : 2 + natom]
    if len(coord_lines) != natom:
        raise ValueError("XYZ file does not contain the declared number of atoms.")

    return _normalize_geometry_lines(coord_lines), title


def default_water_spec(basis: str = "sto-3g") -> MoleculeSpec:
    atom = """
    O 0.000000 0.000000 0.000000
    H 0.000000 -0.757160 0.586260
    H 0.000000 0.757160 0.586260
    """
    return MoleculeSpec(
        atom=_normalize_geometry_lines(atom.splitlines()),
        basis=normalize_basis_name(basis),
        charge=0,
        spin=0,
        unit="Angstrom",
```

```
        title="H2O",
    )
```

这里的两段代码分别负责：

- 从 XYZ 文件读取几何；
- 在用户不提供任何输入时，自动构造一个标准水分子。

read_xyz_geometry

它按照 XYZ 的常见格式做：

1. 第一行读原子数；
2. 第二行读标题；
3. 后面读坐标行；
4. 最后交给 `_normalize_geometry_lines` 做进一步规范化。

default_water_spec

这段代码非常有教学味道，因为它让你在完全不输入任何文件时，也能马上跑出一个真实分子结果。这样初学者第一次运行程序的门槛会低很多。

第六章　RHF 理论：从 Hartree 走向真正可运行的自洽场

6.1　为什么 Hartree 不够

Hartree 近似把总波函数写成轨道乘积：

$$\Psi_H = \prod_i \phi_i(\mathbf{x}_i). \tag{6.1}$$

它的问题是：电子是费米子，而这个乘积形式没有自动满足交换反对称性。

Hartree–Fock 通过 Slater 行列式修正了这一点，因此在平均场层次上更加物理可靠。

6.2　RHF 的矩阵方程

在闭壳层 RHF 中，我们最终要求解的是

$$\mathbf{FC} = \mathbf{SC}\varepsilon. \tag{6.2}$$

这里最重要的对象分别是：

- $\mathbf{S}$：AO 重叠矩阵；
- $\mathbf{h}$：一电子哈密顿量矩阵；
- $\mathbf{D}$：密度矩阵；
- $\mathbf{F}$：Fock 矩阵；
- $\mathbf{C}$：分子轨道系数矩阵；
- ε：轨道能量。

程序中所有 RHF 代码，本质上都在围绕这六个对象展开。

第七章　把 RHF 源码逐段讲透：rhf.py

7.1　结果对象为什么先定义成 dataclass

```python
@dataclass
class RHFResult:
    energy: float
    electronic_energy: float
    nuclear_repulsion: float
    orbital_energies: np.ndarray
    coefficients: np.ndarray
    density: np.ndarray
    iterations: int
    history: list[float]
```

`RHFResult` 是一个结果容器。它把 RHF 输出中最重要的量集中在一起：

- `energy`：总能量
- `electronic_energy`：电子能
- `nuclear_repulsion`：核排斥能
- `orbital_energies`：轨道能
- `coefficients`：分子轨道系数
- `density`：最终密度矩阵
- `iterations`：SCF 迭代步数
- `history`：每一步总能量

为什么这样设计很重要？因为程序跑完之后，你不只想打印一个总能量，而是希望保留后面模块继续需要的全部信息。

如果你以前还没接触过 Python 类，那现在可以先把 `RHFResult` 理解成：

一个带名字的结果盒子，里面整齐地装着 RHF 算完之后最重要的所有量。

7.2 DIIS 缓冲器是怎么设计的

```python
@dataclass
class DIISHelper:
    max_vectors: int = 6

    def __post_init__(self) -> None:
        self.fock_matrices: list[np.ndarray] = []
        self.error_matrices: list[np.ndarray] = []

    def push(self, fock: np.ndarray, error: np.ndarray) -> None:
        self.fock_matrices.append(fock.copy())
        self.error_matrices.append(error.copy())
        if len(self.fock_matrices) > self.max_vectors:
            self.fock_matrices.pop(0)
            self.error_matrices.pop(0)

    def extrapolate(self) -> np.ndarray:
        count = len(self.fock_matrices)
        if count < 2:
            return self.fock_matrices[-1]

        b_matrix = np.empty((count + 1, count + 1))
        b_matrix[:-1, :-1] = np.array(
            [
                [np.vdot(err_i, err_j).real for err_j in self.error_matrices]
                for err_i in self.error_matrices
            ]
        )
        b_matrix[-1, :-1] = -1.0
        b_matrix[:-1, -1] = -1.0
        b_matrix[-1, -1] = 0.0

        rhs = np.zeros(count + 1)
        rhs[-1] = -1.0

        try:
            coefficients = np.linalg.solve(b_matrix, rhs)[:-1]
        except np.linalg.LinAlgError:
            return self.fock_matrices[-1]

        return sum(coeff * fock for coeff, fock in zip(coefficients, self.fock_matrices,
            strict=True))
```

这段代码对初学者来说有点“吓人”，但其实可以拆得很清楚。

__post_init__

这里建立了两个列表：

- `fock_matrices`
- `error_matrices`

它们的意义是：保留最近若干步的 Fock 和误差，供 DIIS 做线性组合。

push

每次 SCF 新产生一个 Fock，就把它和对应误差都压进去。如果数量超过上限，就把最旧的弹掉。

这说明 DIIS 不是在用“全部历史”，而是在用“最近的一个滑动窗口”。

这里顺便也能练三条基础语法：

- `append(...)`：把新元素加到列表末尾；
- `len(...)`：取列表长度；
- `pop(0)`：删除最前面的旧元素。

extrapolate

这一段是 DIIS 的数学核心。它建立了一个带约束的线性方程组：

$$\begin{pmatrix} \mathbf{B} & -\mathbf{1} \\ -\mathbf{1}^T & 0 \end{pmatrix} \begin{pmatrix} \mathbf{c} \\ \lambda \end{pmatrix} = \begin{pmatrix} \mathbf{0} \\ -1 \end{pmatrix}. \tag{7.1}$$

在代码里：

- `b_matrix` 就是这个大矩阵；
- `rhs` 是右端项；
- `coefficients` 是求出来的 DIIS 线性组合系数。

如果矩阵奇异，代码会回退到“直接使用最新 Fock”，这也是很典型的稳健性设计。

7.3 把 MoleculeSpec 变成 PySCF 分子对象

```
def build_molecule(spec: MoleculeSpec) -> gto.Mole:
    mol = gto.Mole()
    mol.atom = spec.atom
    mol.unit = spec.unit
```

```
    mol.basis = spec.basis
    mol.charge = spec.charge
    mol.spin = spec.spin
    mol.build()
    return mol
```

这段非常关键，因为从这里开始，项目自己的输入结构正式转化为 PySCF 可识别对象。其中：

- `mol.atom` 接收坐标字符串；
- `mol.unit` 接收单位；
- `mol.basis` 接收基组；
- `mol.charge` 和 `mol.spin` 给出电子数约束；
- `mol.build()` 真正完成分子构建。

如果你第一次见到 `mol.atom = spec.atom` 这种写法，可以把它简单理解为：

> 把 `spec` 里记录的信息，一项一项抄进 `mol` 这个对象里。

7.4 为什么还要有 build_rhf_reference_mf

```
def build_rhf_reference_mf(mol: gto.Mole, rhf_result: RHFResult) -> tuple[scf.RHF, np.
    ndarray]:
    mf = scf.RHF(mol)
    nocc = mol.nelectron // 2
    mo_occ = np.zeros(mol.nao_nr())
    mo_occ[:nocc] = 2.0
    mf.mo_coeff = rhf_result.coefficients
    mf.mo_energy = rhf_result.orbital_energies
    mf.mo_occ = mo_occ
    mf.e_tot = rhf_result.energy
    mf.converged = True
    return mf, mo_occ
```

这一段不是 RHF 主循环的一部分，而是为后续模块准备的“桥接器”。它的作用是：

> 把我们自己算出来的 RHF 结果，重新装配成一个 PySCF 的 `mf` 对象。

这样做的好处是：

- 前面的 SCF 我们自己写，便于教学；
- 后面的梯度、Hessian、相关方法又可以继续复用 PySCF 生态。

7.5 正交化、密度矩阵和 Fock 矩阵

```python
def symmetric_orthogonalization(overlap: np.ndarray) -> np.ndarray:
    eigenvalues, eigenvectors = np.linalg.eigh(overlap)
    return eigenvectors @ np.diag(eigenvalues ** -0.5) @ eigenvectors.T


def build_density(coefficients: np.ndarray, nocc: int) -> np.ndarray:
    occupied = coefficients[:, :nocc]
    return 2.0 * occupied @ occupied.T


def build_fock(h_core: np.ndarray, eri: np.ndarray, density: np.ndarray) -> np.ndarray:
    coulomb = np.einsum("ls,mnls->mn", density, eri, optimize=True)
    exchange = np.einsum("ls,mlns->mn", density, eri, optimize=True)
    return h_core + coulomb - 0.5 * exchange


def compute_diis_error(fock: np.ndarray, density: np.ndarray, overlap: np.ndarray, x: np.ndarray) -> np.ndarray:
    commutator = fock @ density @ overlap - overlap @ density @ fock
    return x.T @ commutator @ x


def diagonalize_fock(fock: np.ndarray, x: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
    fock_prime = x.T @ fock @ x
    orbital_energies, coeff_prime = np.linalg.eigh(fock_prime)
    return orbital_energies, x @ coeff_prime
```

这一小段几乎就是 RHF 的数学骨架。

symmetric_orthogonalization

它把重叠矩阵 $\mathbf{S}$ 做本征分解：

$$\mathbf{S} = \mathbf{U}\boldsymbol{\Lambda}\mathbf{U}^T, \tag{7.2}$$

然后构造

$$\mathbf{X} = \mathbf{U}\boldsymbol{\Lambda}^{-1/2}\mathbf{U}^T. \tag{7.3}$$

这样我们就能把广义本征值问题变成普通本征值问题。

build_density

闭壳层 RHF 中：

$$\mathbf{D} = 2\mathbf{C}_{\text{occ}}\mathbf{C}_{\text{occ}}^T. \tag{7.4}$$

代码里直接把前 nocc 个轨道取出来，相乘后再乘 2。

这里的切片

```
coefficients[:, :nocc]
```

请你一定练熟。它表示：

- 所有行都保留；
- 列只取前 nocc 个。

这正好对应“取出占据轨道”。

build_fock

这一段就是 AO 基中的

$$\mathbf{F} = \mathbf{h} + \mathbf{J} - \frac{1}{2}\mathbf{K}. \tag{7.5}$$

初学者尤其要注意：这里的 einsum 不是“炫技”，而是把多重指标求和直接写出来的一种非常自然的方式。

如果你目前还完全看不懂 einsum，不要慌。你现在只要先抓住这一层：

> build_fock 的输入是 h_core、双电子积分和密度，输出是新的 Fock 矩阵。

先把函数的输入输出角色搞清楚，再回头慢慢理解内部张量缩并，会更适合零基础学习。

compute_diis_error

它计算的是

$$\mathbf{e} = \mathbf{X}^T(\mathbf{FDS} - \mathbf{SDF})\mathbf{X}. \tag{7.6}$$

当 SCF 真正收敛时，这个误差矩阵应接近零。

diagonalize_fock

它先把 Fock 变换到正交基：

$$\mathbf{F}' = \mathbf{X}^T\mathbf{F}\mathbf{X}, \tag{7.7}$$

然后再对角化得到轨道能和系数。

7.6 为什么 RHF 必须先检查闭壳层条件

```python
def _validate_closed_shell_rhf(mol: gto.Mole) -> None:
    if mol.spin != 0:
        raise ValueError(
```

```
            "This teaching example implements closed-shell RHF only. "
            "Please use a system with spin = 0."
        )
    if mol.nelectron % 2 != 0:
        raise ValueError(
            "Closed-shell RHF requires an even number of electrons after charge is applied. "
        )
```

RHF 假设的是每个空间轨道有一对自旋相反电子，因此它要求：

- spin = 0
- 总电子数是偶数

这段输入检查其实是在保护整个程序的物理前提。

7.7 最关键的函数：run_rhf

```
def run_rhf(
    mol: gto.Mole,
    max_iter: int = 100,
    e_tol: float = 1.0e-10,
    d_tol: float = 1.0e-8,
    use_diis: bool = True,
    diis_space: int = 6,
) -> RHFResult:
    _validate_closed_shell_rhf(mol)

    s = mol.intor("int1e_ovlp")
    t = mol.intor("int1e_kin")
    v = mol.intor("int1e_nuc")
    eri = mol.intor("int2e")
    h_core = t + v
    e_nuc = mol.energy_nuc()

    nocc = mol.nelectron // 2
    x = symmetric_orthogonalization(s)
    diis_helper = DIISHelper(max_vectors=diis_space)

    _, coeff = diagonalize_fock(h_core, x)
    density = build_density(coeff, nocc)

    previous_energy = None
    history: list[float] = []

```

```
    for iteration in range(1, max_iter + 1):
        fock = build_fock(h_core, eri, density)
        if use_diis:
            diis_error = compute_diis_error(fock, density, s, x)
            diis_helper.push(fock, diis_error)
            fock_to_diagonalize = diis_helper.extrapolate()
        else:
            fock_to_diagonalize = fock

        orbital_energies, coeff = diagonalize_fock(fock_to_diagonalize, x)
        new_density = build_density(coeff, nocc)
        new_fock = build_fock(h_core, eri, new_density)

        e_elec = 0.5 * np.sum(new_density * (h_core + new_fock))
        e_total = e_elec + e_nuc
        history.append(e_total)

        density_change = np.linalg.norm(new_density - density)
        if previous_energy is not None and abs(e_total - previous_energy) < e_tol and
            density_change < d_tol:
            orbital_energies, coeff = diagonalize_fock(new_fock, x)
            return RHFResult(
                energy=float(e_total),
                electronic_energy=float(e_elec),
                nuclear_repulsion=float(e_nuc),
                orbital_energies=orbital_energies,
                coefficients=coeff,
                density=new_density,
                iterations=iteration,
                history=history,
            )

        density = new_density
        previous_energy = e_total

    raise RuntimeError("SCF did not converge within the iteration limit.")
```

这是本项目第一重要的函数。你应该把它当成“RHF 的完整算法说明书”。

第一步：拿积分

代码一开始：

- `s = mol.intor("int1e_ovlp")`：重叠积分
- `t = mol.intor("int1e_kin")`：动能积分

- `v = mol.intor("int1e_nuc")`：核吸引积分
- `eri = mol.intor("int2e")`：双电子积分

这是从抽象公式走向可计算矩阵的关键一步。

第二步：构造一电子哈密顿量

$$\mathbf{h} = \mathbf{T} + \mathbf{V}. \tag{7.8}$$

代码里就是

```
h_core = t + v
```

第三步：给一个初始密度

程序先对角化 `h_core`，再用得到的占据轨道构造初始密度。这是一种很常见的初猜。

这里最好在脑子里把动作拆成两步：

1. 先猜一套轨道；
2. 再由轨道生成一套密度。

也就是说，程序并不是“凭空猜密度”，而是“先通过一电子哈密顿量猜轨道，再由轨道推出密度”。

第四步：进入 SCF 循环

循环里的逻辑必须真正理解：

1. 用旧密度生成 Fock；
2. 如果启用 DIIS，就用误差矩阵做外推；
3. 对角化得到新轨道；
4. 用新轨道生成新密度；
5. 用新密度再生成一次新 Fock；
6. 计算新总能量；
7. 检查能量和密度是否收敛。

对第一次接触数值迭代的读者来说，可以把整个 SCF 循环先想成一句特别朴素的话：

> 先拿当前答案生成一个更好的答案，再拿更好的答案继续修正自己，直到变化足够小。

第五步：为什么能量公式前面有 1/2

$$E_{\text{elec}} = \frac{1}{2}\sum_{\mu\nu} D_{\mu\nu}(h_{\mu\nu} + F_{\mu\nu}). \tag{7.9}$$

这个 $\frac{1}{2}$ 是为了避免双计数双电子相互作用。很多初学者第一次实现 RHF 时最容易在这里写错。

第六步：返回 RHFResult

一旦收敛，函数不会只返回一个数，而是返回一个结构化结果对象。这使得后面：

- MP2
- CCSD
- 频率分析

都可以继续复用 RHF 轨道与密度。

第八章　RHF 跑通之后，为什么还要学 UHF

8.1　闭壳层体系和开壳层体系

RHF 很适合像水分子这种闭壳层体系，但遇到 OH 自由基这样的开壳层体系时，α 和 β 电子数不再相同。

如果你还强行用 RHF，就会把体系的真实自旋结构压扁。

8.2　UHF 的基本思想

UHF 的核心思想只有一句话：

让 α 和 β 电子拥有各自独立的轨道、密度矩阵和 Fock 矩阵。

数学上它对应：

$$\mathbf{F}^{\alpha} = \mathbf{h} + \mathbf{J}[\mathbf{D}^{\alpha} + \mathbf{D}^{\beta}] - \mathbf{K}[\mathbf{D}^{\alpha}], \tag{8.1}$$

$$\mathbf{F}^{\beta} = \mathbf{h} + \mathbf{J}[\mathbf{D}^{\alpha} + \mathbf{D}^{\beta}] - \mathbf{K}[\mathbf{D}^{\beta}]. \tag{8.2}$$

和 RHF 相比，最大的变化就是：一个密度矩阵变成了两个。

第九章　把 UHF 源码逐段讲透：uhf.py

9.1　结果对象比 RHF 多了什么

```python
@dataclass
class UHFResult:
    energy: float
    electronic_energy: float
    nuclear_repulsion: float
    orbital_energies_alpha: np.ndarray
    orbital_energies_beta: np.ndarray
    coefficients_alpha: np.ndarray
    coefficients_beta: np.ndarray
    density_alpha: np.ndarray
    density_beta: np.ndarray
    iterations: int
    history: list[float]
    nalpha: int
    nbeta: int
    s2: float
    expected_s2: float
    spin_contamination: float
```

`UHFResult` 比 `RHFResult` 多出来的主要有：

- `orbital_energies_alpha`
- `orbital_energies_beta`
- `coefficients_alpha`
- `coefficients_beta`
- `density_alpha`
- `density_beta`
- `nalpha`, `nbeta`
- `s2`, `expected_s2`, `spin_contamination`

这正好对应 UHF 相比 RHF 的两个新主题：

1. 自旋分辨的轨道与密度；
2. 自旋污染分析。

9.2 单自旋密度矩阵

```
def build_spin_density(coefficients: np.ndarray, nocc: int) -> np.ndarray:
    if nocc == 0:
        return np.zeros((coefficients.shape[0], coefficients.shape[0]))
    occupied = coefficients[:, :nocc]
    return occupied @ occupied.T
```

这段函数和 RHF 的 `build_density` 非常像，但不再乘 2。原因很简单：

> 这里一个密度矩阵只对应一个自旋分量，不再是成对占据。

所以：

$$\mathbf{D}^{\alpha} = \mathbf{C}^{\alpha}_{\text{occ}}(\mathbf{C}^{\alpha}_{\text{occ}})^T, \tag{9.1}$$

$$\mathbf{D}^{\beta} = \mathbf{C}^{\beta}_{\text{occ}}(\mathbf{C}^{\beta}_{\text{occ}})^T. \tag{9.2}$$

所以从代码结构上看，RHF 和 UHF 的关键差异之一就是：

- RHF 只有一个密度矩阵，而且要乘 2；
- UHF 有两个自旋分辨的密度矩阵，而且都不乘 2。

9.3 UHF 的 Fock 矩阵是如何构造的

```
def build_uhf_fock(
    h_core: np.ndarray,
    eri: np.ndarray,
    density_alpha: np.ndarray,
    density_beta: np.ndarray,
) -> tuple[np.ndarray, np.ndarray]:
    density_total = density_alpha + density_beta
    coulomb = np.einsum("ls,mnls->mn", density_total, eri, optimize=True)
    exchange_alpha = np.einsum("ls,mlns->mn", density_alpha, eri, optimize=True)
    exchange_beta = np.einsum("ls,mlns->mn", density_beta, eri, optimize=True)
    fock_alpha = h_core + coulomb - exchange_alpha
    fock_beta = h_core + coulomb - exchange_beta
    return fock_alpha, fock_beta
```

这一段是 UHF 的核心公式翻译。

为什么 Coulomb 用总密度

电子所感受到的静电排斥，并不区分对方是 α 还是 β，所以 Coulomb 项使用总密度：

$$\mathbf{D}^{\text{tot}} = \mathbf{D}^{\alpha} + \mathbf{D}^{\beta}. \tag{9.3}$$

为什么 Exchange 要分自旋

交换作用只发生在相同自旋电子之间，所以：

- α Fock 只减去 α 的交换；
- β Fock 只减去 β 的交换。

这就是 UHF 与 RHF 在结构上的根本差异。

9.4 为什么要把 alpha 和 beta 拼成块矩阵

```python
def combine_spin_blocks(matrix_alpha: np.ndarray, matrix_beta: np.ndarray) -> np.ndarray:
    zeros = np.zeros_like(matrix_alpha)
    return np.block([[matrix_alpha, zeros], [zeros, matrix_beta]])


def split_spin_blocks(matrix: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
    nbf = matrix.shape[0] // 2
    return matrix[:nbf, :nbf], matrix[nbf:, nbf:]
```

很多初学者第一次看到 `combine_spin_blocks` 会问：既然已经有两个 Fock 了，为什么还要拼起来？

答案是：因为项目里想继续复用 RHF 中已经写好的 `DIISHelper`。而那个 DIIS 缓冲器期望接收的是“单个矩阵”。于是这里把

$$\mathbf{F}^{\alpha}, \mathbf{F}^{\beta} \tag{9.4}$$

拼成一个块对角矩阵：

$$\begin{pmatrix} \mathbf{F}^{\alpha} & 0 \\ 0 & \mathbf{F}^{\beta} \end{pmatrix}. \tag{9.5}$$

这是一种非常漂亮的复用技巧。

从 Python 语法角度看，这里最值得你认识的是 `np.block(...)`。你可以先把它理解成：

> 按照指定布局，把几个小矩阵拼成一个更大的矩阵。

9.5 输入检查和电子数分配

```python
def _validate_uhf_inputs(mol: gto.Mole) -> tuple[int, int]:
    nalpha, nbeta = mol.nelec
    if nalpha < 0 or nbeta < 0:
        raise ValueError("Invalid alpha/beta electron counts for UHF.")
    return nalpha, nbeta
```

PySCF 会根据总电荷和 spin 给出

$$(N_\alpha, N_\beta). \tag{9.6}$$

这个函数做的只是最基本的合法性检查。它提醒我们：

> UHF 的输入核心不是“spin 标签本身”，而是最终的 alpha/beta 电子数。

9.6 $\langle S^2 \rangle$ 是怎么计算的

```python
def compute_uhf_s2(
    overlap: np.ndarray,
    coeff_alpha: np.ndarray,
    coeff_beta: np.ndarray,
    nalpha: int,
    nbeta: int,
) -> tuple[float, float, float]:
    occ_alpha = coeff_alpha[:, :nalpha]
    occ_beta = coeff_beta[:, :nbeta]
    if nalpha == 0 or nbeta == 0:
        overlap_occ = 0.0
    else:
        spin_overlap = occ_alpha.T @ overlap @ occ_beta
        overlap_occ = float(np.sum(np.abs(spin_overlap) ** 2))

    sz = 0.5 * (nalpha - nbeta)
    s2 = sz * (sz + 1.0) + nbeta - overlap_occ
    expected_s2 = 0.5 * mol_spin_from_counts(nalpha, nbeta) * (0.5 * mol_spin_from_counts(
        nalpha, nbeta) + 1.0)
    return s2, expected_s2, s2 - expected_s2


def mol_spin_from_counts(nalpha: int, nbeta: int) -> int:
    return nalpha - nbeta
```

这是 UHF 里最有“量子化学味道”的一段代码。

为什么要算 $\langle S^2 \rangle$

因为 UHF 虽然能处理开壳层，但它的单行列式解不一定是严格自旋本征态，所以会出现自旋污染。

代码在做什么

它先取出占据的 α 和 β 轨道，然后计算占据轨道之间的重叠：

$$\mathbf{S}_{\text{occ}} = (\mathbf{C}_{\text{occ}}^{\alpha})^T \mathbf{S} \mathbf{C}_{\text{occ}}^{\beta}. \tag{9.7}$$

再根据公式

$$\langle S^2 \rangle = S_z(S_z+1) + N_\beta - \sum_{ij} \left| S_{ij}^{\alpha\beta} \right|^2 \tag{9.8}$$

求出实际的 $\langle S^2 \rangle$。

如果你读代码时觉得这一段比较抽象，可以先只抓住三步：

1. 先拿出占据的 alpha 轨道；
2. 再拿出占据的 beta 轨道；
3. 然后用重叠矩阵去计算它们之间的重叠。

expected_s2 和 spin_contamination

- `expected_s2` 是理想自旋本征态该有的值；
- `spin_contamination` 是实际值减理想值。

这三个量一起输出，才构成真正有意义的开壳层结果分析。

9.7 最关键的函数：run_uhf

```python
def run_uhf(
    mol: gto.Mole,
    max_iter: int = 100,
    e_tol: float = 1.0e-10,
    d_tol: float = 1.0e-8,
    use_diis: bool = True,
    diis_space: int = 6,
) -> UHFResult:
    nalpha, nbeta = _validate_uhf_inputs(mol)

    s = mol.intor("int1e_ovlp")
    t = mol.intor("int1e_kin")
```

```
    v = mol.intor("int1e_nuc")
    eri = mol.intor("int2e")
    h_core = t + v
    e_nuc = mol.energy_nuc()

    x = symmetric_orthogonalization(s)
    diis_helper = DIISHelper(max_vectors=diis_space)

    orbital_energies_alpha, coeff_alpha = diagonalize_fock(h_core, x)
    orbital_energies_beta, coeff_beta = diagonalize_fock(h_core, x)
    density_alpha = build_spin_density(coeff_alpha, nalpha)
    density_beta = build_spin_density(coeff_beta, nbeta)

    previous_energy = None
    history: list[float] = []

    for iteration in range(1, max_iter + 1):
        fock_alpha, fock_beta = build_uhf_fock(h_core, eri, density_alpha, density_beta)
        if use_diis:
            error_alpha = compute_diis_error(fock_alpha, density_alpha, s, x)
            error_beta = compute_diis_error(fock_beta, density_beta, s, x)
            diis_helper.push(
                combine_spin_blocks(fock_alpha, fock_beta),
                combine_spin_blocks(error_alpha, error_beta),
            )
            fock_alpha_to_diagonalize, fock_beta_to_diagonalize = split_spin_blocks(
                diis_helper.extrapolate()
            )
        else:
            fock_alpha_to_diagonalize = fock_alpha
            fock_beta_to_diagonalize = fock_beta

        orbital_energies_alpha, coeff_alpha = diagonalize_fock(fock_alpha_to_diagonalize, x
            )
        orbital_energies_beta, coeff_beta = diagonalize_fock(fock_beta_to_diagonalize, x)

        new_density_alpha = build_spin_density(coeff_alpha, nalpha)
        new_density_beta = build_spin_density(coeff_beta, nbeta)
        new_fock_alpha, new_fock_beta = build_uhf_fock(
            h_core,
            eri,
            new_density_alpha,
            new_density_beta,
        )

        electronic_energy = 0.5 * (
```

```
            np.sum((new_density_alpha + new_density_beta) * h_core)
            + np.sum(new_density_alpha * new_fock_alpha)
            + np.sum(new_density_beta * new_fock_beta)
        )
        total_energy = electronic_energy + e_nuc
        history.append(total_energy)

        density_change = np.sqrt(
            np.linalg.norm(new_density_alpha - density_alpha) ** 2
            + np.linalg.norm(new_density_beta - density_beta) ** 2
        )
        if previous_energy is not None and abs(total_energy - previous_energy) < e_tol and density_change < d_tol:
            orbital_energies_alpha, coeff_alpha = diagonalize_fock(new_fock_alpha, x)
            orbital_energies_beta, coeff_beta = diagonalize_fock(new_fock_beta, x)
            s2, expected_s2, spin_contamination = compute_uhf_s2(
                s,
                coeff_alpha,
                coeff_beta,
                nalpha,
                nbeta,
            )
            return UHFResult(
                energy=float(total_energy),
                electronic_energy=float(electronic_energy),
                nuclear_repulsion=float(e_nuc),
                orbital_energies_alpha=orbital_energies_alpha,
                orbital_energies_beta=orbital_energies_beta,
                coefficients_alpha=coeff_alpha,
                coefficients_beta=coeff_beta,
                density_alpha=new_density_alpha,
                density_beta=new_density_beta,
                iterations=iteration,
                history=history,
                nalpha=nalpha,
                nbeta=nbeta,
                s2=float(s2),
                expected_s2=float(expected_s2),
                spin_contamination=float(spin_contamination),
            )

        density_alpha = new_density_alpha
        density_beta = new_density_beta
        previous_energy = total_energy

    raise RuntimeError("UHF did not converge within the iteration limit.")
```

你应该把这段代码看成“RHF 的双自旋扩展版”。

第一阶段：准备积分与初猜

它和 RHF 一样先取：

- `s`
- `t`
- `v`
- `eri`
- `h_core`

接着分别给 α 和 β 做初始对角化，并构造两个自旋密度矩阵。

对初学者来说，这里最重要的理解是：

> UHF 并不是完全推翻 RHF，而是在 RHF 框架上复制出两条自旋通道。

第二阶段：SCF 循环

循环里发生的是：

1. 根据当前 α/β 密度构造两套 Fock；
2. 如果启用 DIIS，就把它们拼成块矩阵做外推；
3. 分别对角化得到新的 α/β 轨道；
4. 分别更新 α/β 密度；
5. 用新密度重新生成 Fock；
6. 计算总电子能和总能量；
7. 用联合密度变化判断收敛。

建议你第一次读这一段时，真的在纸上把变量抄下来：

- `density_alpha`
- `density_beta`
- `fock_alpha`
- `fock_beta`
- `new_density_alpha`
- `new_density_beta`

当你能把这些变量分别对应什么物理量说清楚时，UHF 其实已经懂了一大半。

第三阶段：UHF 总能量公式

代码里用了

$$E_{\text{elec}} = \frac{1}{2}\left[\mathrm{Tr}\left((\mathbf{D}^{\alpha}+\mathbf{D}^{\beta})\mathbf{h}\right) + \mathrm{Tr}\left(\mathbf{D}^{\alpha}\mathbf{F}^{\alpha}\right) + \mathrm{Tr}\left(\mathbf{D}^{\beta}\mathbf{F}^{\beta}\right)\right]. \tag{9.9}$$

它和 RHF 的区别在于：

- 一电子项由总密度给出；
- Fock 项要按自旋分开处理。

第四阶段：为什么收敛后还要重新对角化一次

这和 RHF 一样，是为了确保返回结果中的轨道能与最终 Fock 一致。

9.8 为什么还有 build_uhf_reference_mf

```
def build_uhf_reference_mf(mol: gto.Mole, uhf_result: UHFResult) -> tuple[scf.UHF, tuple[
    np.ndarray, np.ndarray]]:
    mf = scf.UHF(mol)
    mo_occ_alpha = np.zeros(mol.nao_nr())
    mo_occ_beta = np.zeros(mol.nao_nr())
    mo_occ_alpha[: uhf_result.nalpha] = 1.0
    mo_occ_beta[: uhf_result.nbeta] = 1.0
    mf.mo_coeff = (uhf_result.coefficients_alpha, uhf_result.coefficients_beta)
    mf.mo_energy = (uhf_result.orbital_energies_alpha, uhf_result.orbital_energies_beta)
    mf.mo_occ = (mo_occ_alpha, mo_occ_beta)
    mf.e_tot = uhf_result.energy
    mf.converged = True
    return mf, (mo_occ_alpha, mo_occ_beta)
```

它和 RHF 版本作用完全一样：把我们自己求得的 UHF 结果重新包装成 PySCF 的 `UHF` 对象，供后续模块继续使用。

第十章　把“从输入到输出”的整条路径串起来

10.1　RHF 模式下，程序一步一步经历了什么

当你运行

```
python3 rhf_sto3g_water.py --method rhf
```

程序的实际执行路径是：

1. 进入 `rhf_sto3g_water.py`
2. 调用 `simple_hf.cli.main()`
3. 在 `build_parser()` 中构造参数解释器
4. 在 `build_spec_from_args()` 中得到 `MoleculeSpec`
5. 用 `build_molecule(spec)` 构造 `mol`
6. 在 `run_rhf(mol)` 中真正执行 SCF
7. 返回 `RHFResult`
8. 用打印函数把结果显示出来

10.2　UHF 模式下，多出来了什么

当你运行

```
python3 rhf_sto3g_water.py --method uhf --xyz examples/oh_radical.xyz --spin 1
```

前半段路径完全一样。真正发生变化的是求解器：

- 从 `run_rhf` 变成 `run_uhf`
- 从一个密度矩阵变成两个密度矩阵
- 从一个 Fock 变成两个 Fock
- 从一个轨道能数组变成 alpha/beta 两套轨道能
- 最后还多输出 $\langle S^2 \rangle$

第十一章　如果你要自己从零重写这一部分，应该按什么顺序

11.1　最推荐的复现顺序

对于零基础读者，我最推荐的顺序是：

1. 先自己写一个最小 `MoleculeSpec`
2. 再写 `build_molecule`
3. 再写 `symmetric_orthogonalization`
4. 再写 `build_density`
5. 再写 `build_fock`
6. 再写不带 DIIS 的 `run_rhf`
7. 跑通水分子
8. 再加入 `DIISHelper`
9. 再把 RHF 扩展成 UHF
10. 最后再加入 $\langle S^2 \rangle$ 分析

千万不要上来就试图一口气把 `geometry.py`、`cli.py`、`rhf.py`、`uhf.py` 全抄下来，那样很容易看花。

11.2　每完成一步，你应该检查什么

完成 RHF 后

你应该能回答：

- 重叠矩阵为什么要正交化？
- 密度矩阵为什么是 $2C_{\mathrm{occ}}C_{\mathrm{occ}}^T$？
- 为什么能量公式前面有 $\frac{1}{2}$？

完成 DIIS 后

你应该能回答：

- 为什么要缓存最近几步的 Fock？
- 误差矩阵为什么用对易子表示？
- 为什么 DIIS 有时会回退到最新 Fock？

完成 UHF 后

你应该能回答：

- 为什么 Coulomb 用总密度，而 Exchange 要分自旋？
- 为什么 UHF 可能有自旋污染？
- 为什么一个开壳层体系不能直接继续用 RHF？

第二部分

第二部分：从相关能到结构分析

第十二章　MP2 与 UMP2：第一次真正看到“电子相关”

本章目标

这一章要解决两个问题。第一，为什么 RHF/UHF 之后还不够；第二，`mp2.py` 和 `ump2.py` 到底是怎样把“占据轨道 + 虚轨道 + 轨道能分母”拼成相关能的。

12.1　为什么 RHF/UHF 之后先学 MP2

Hartree–Fock 的核心优点，是把多电子问题压缩成了可迭代的单行列式问题；但它仍然缺少电子相关。最直观地说，HF 只让每个电子看见别的电子的**平均场**，没有完全描述“电子会相互躲开”这件事。

MP2 是最经典的第一步后 HF 方法。它在 HF 参考态上做二阶微扰修正，公式可以写成

$$E_{\mathrm{MP2}} = \sum_{ijab} \frac{\langle ia|jb\rangle \left(2\langle ia|jb\rangle - \langle ib|ja\rangle\right)}{\varepsilon_i + \varepsilon_j - \varepsilon_a - \varepsilon_b}. \tag{12.1}$$

这一式子对于初学者看起来很吓人，但代码实现的核心其实只有四步：

1. 从 RHF 结果里取出占据轨道和虚轨道；
2. 把 AO 基底的双电子积分变换到 MO 基底的 $ovov$ 形式；
3. 组装能量分母；
4. 做一次大求和。

12.2　先把 `mp2.py` 整体看一遍

```python
from __future__ import annotations

from dataclasses import dataclass

import numpy as np
from pyscf import gto

```

```python
from .rhf import RHFResult


@dataclass
class MP2Result:
    rhf_energy: float
    mp2_correlation_energy: float
    total_energy: float
    orbital_energies: np.ndarray
    nocc: int
    nvir: int


def transform_ao_eri_to_ovov(
    ao_eri: np.ndarray,
    occ_coeff: np.ndarray,
    vir_coeff: np.ndarray,
) -> np.ndarray:
    return np.einsum(
        "pqrs,pi,qa,rj,sb->iajb",
        ao_eri,
        occ_coeff,
        vir_coeff,
        occ_coeff,
        vir_coeff,
        optimize=True,
    )


def run_mp2(mol: gto.Mole, rhf_result: RHFResult) -> MP2Result:
    nocc = mol.nelectron // 2
    nmo = rhf_result.coefficients.shape[1]
    nvir = nmo - nocc
    if nvir <= 0:
        raise ValueError("MP2 requires at least one virtual orbital.")

    occ_coeff = rhf_result.coefficients[:, :nocc]
    vir_coeff = rhf_result.coefficients[:, nocc:]
    eps_occ = rhf_result.orbital_energies[:nocc]
    eps_vir = rhf_result.orbital_energies[nocc:]

    ao_eri = mol.intor("int2e")
    eri_ovov = transform_ao_eri_to_ovov(ao_eri, occ_coeff, vir_coeff)
    exchange_ovov = eri_ovov.transpose(0, 3, 2, 1)

    denominators = (
```

```
        eps_occ[:, None, None, None]
        + eps_occ[None, None, :, None]
        - eps_vir[None, :, None, None]
        - eps_vir[None, None, None, :]
    )

    correlation_energy = np.sum(
        eri_ovov * (2.0 * eri_ovov - exchange_ovov) / denominators
    )
    total_energy = rhf_result.energy + correlation_energy

    return MP2Result(
        rhf_energy=float(rhf_result.energy),
        mp2_correlation_energy=float(correlation_energy),
        total_energy=float(total_energy),
        orbital_energies=rhf_result.orbital_energies.copy(),
        nocc=nocc,
        nvir=nvir,
    )
```

这个文件的结构非常短，但非常典型

- `MP2Result` 负责把 RHF 能、MP2 相关能和总能打包。
- `transform_ao_eri_to_ovov()` 负责积分变换。
- `run_mp2()` 负责从 RHF 结果生成 MP2 能量。

这类文件特别适合初学者学习，因为它几乎把“理论对象”和“程序对象”的映射关系摆在明面上了。

12.3 transform_ao_eri_to_ovov() 到底在做什么

最难的一行是：

```
np.einsum("pqrs,pi,qa,rj,sb->iajb", ao_eri, occ_coeff, vir_coeff,
        occ_coeff, vir_coeff, optimize=True)
```

如果你是第一次学 Python/NumPy，请先不要试图一次把整个 Einstein 求和记住。你只需要知道：

- `ao_eri[p,q,r,s]` 是 AO 基底双电子积分；
- `occ_coeff[p,i]` 把 AO 指标 p 投影到占据 MO 指标 i；
- `vir_coeff[q,a]` 把 AO 指标 q 投影到虚轨道指标 a；

- 返回的 `iajb` 就是我们在 MP2 公式里真正要用的积分块。

也就是说，这一行代码不是“神秘技巧”，只是把

$$(ia|jb) = \sum_{pqrs} C_{pi}^{\mathrm{occ}} C_{qa}^{\mathrm{vir}} C_{rj}^{\mathrm{occ}} C_{sb}^{\mathrm{vir}} (pq|rs) \tag{12.2}$$

直接翻译成了张量收缩。

12.4 `run_mp2()` 应该怎样读

第一步：先分清楚哪些轨道是占据的

`nocc = mol.nelectron // 2` 这一行表示闭壳层 RHF 中，每个空间轨道装 2 个电子，所以占据轨道数就是电子数除以 2。

然后代码用切片取出：

- `coefficients[:, :nocc]` 作为占据轨道系数矩阵；
- `coefficients[:, nocc:]` 作为虚轨道系数矩阵。

你在 RHF 章节里学过，`[:, :nocc]` 的意思就是“所有行，前 `nocc` 列”。

第二步：为什么要把轨道能也分成两段

MP2 分母是

$$\varepsilon_i + \varepsilon_j - \varepsilon_a - \varepsilon_b. \tag{12.3}$$

所以代码里把轨道能分成

- `eps_occ`
- `eps_vir`

这样后面就能靠 NumPy 广播机制自动生成四维分母张量。

第三步：广播是如何把分母拼出来的

```
denominators = (
    eps_occ[:, None, None, None]
    + eps_occ[None, None, :, None]
    - eps_vir[None, :, None, None]
    - eps_vir[None, None, None, :]
)
```

这一段是初学者最容易晕的地方。请慢一点读：

- `eps_occ[:, None, None, None]` 把一维数组变成 i 方向变化的四维张量；
- 第二项让 j 方向变化；
- 第三项让 a 方向变化；
- 第四项让 b 方向变化。

最后 NumPy 会自动把它们拼成 (i, a, j, b) 形状的完整分母。

第四步：交换积分是怎么来的

`exchange_ovov = eri_ovov.transpose(0, 3, 2, 1)` 的作用，是把

$$(ia|jb) \tag{12.4}$$

重排成

$$(ib|ja). \tag{12.5}$$

所以最后一行

```
np.sum(eri_ovov * (2.0 * eri_ovov - exchange_ovov) / denominators)
```

就是 MP2 公式的直接实现。

初学者提醒

MP2 相关能通常是负的，所以总能一般会比 RHF 更低。如果你以后自己重写程序，发现 MP2 相关能大范围为正，第一优先应该检查积分指标顺序和分母符号。

12.5 UMP2：为什么会比 MP2 多出三个相关能分量

```
from __future__ import annotations

from dataclasses import dataclass

import numpy as np
from pyscf import gto

from .uhf import UHFResult


@dataclass
class UMP2Result:
    uhf_energy: float
    correlation_energy_aa: float
    correlation_energy_ab: float
    correlation_energy_bb: float
```

```
    ump2_correlation_energy: float
    total_energy: float
    orbital_energies_alpha: np.ndarray
    orbital_energies_beta: np.ndarray
    nocc_alpha: int
    nocc_beta: int
    nvir_alpha: int
    nvir_beta: int


def transform_ao_eri_to_ovov(
    ao_eri: np.ndarray,
    occ_left: np.ndarray,
    vir_left: np.ndarray,
    occ_right: np.ndarray,
    vir_right: np.ndarray,
) -> np.ndarray:
    return np.einsum(
        "pqrs,pi,qa,rj,sb->iajb",
        ao_eri,
        occ_left,
        vir_left,
        occ_right,
        vir_right,
        optimize=True,
    )


def same_spin_mp2_energy(eri_ovov: np.ndarray, eps_occ: np.ndarray, eps_vir: np.ndarray)
    -> float:
    if eps_occ.size == 0 or eps_vir.size == 0:
        return 0.0

    exchange_ovov = eri_ovov.transpose(0, 3, 2, 1)
    antisymmetrized = eri_ovov - exchange_ovov
    denominators = (
        eps_occ[:, None, None, None]
        + eps_occ[None, None, :, None]
        - eps_vir[None, :, None, None]
        - eps_vir[None, None, None, :]
    )
    return 0.25 * np.sum(antisymmetrized * antisymmetrized / denominators)


def opposite_spin_mp2_energy(
    eri_ovov: np.ndarray,
```

```
    eps_occ_left: np.ndarray,
    eps_occ_right: np.ndarray,
    eps_vir_left: np.ndarray,
    eps_vir_right: np.ndarray,
) -> float:
    if eps_occ_left.size == 0 or eps_occ_right.size == 0 or eps_vir_left.size == 0 or
        eps_vir_right.size == 0:
        return 0.0

    denominators = (
        eps_occ_left[:, None, None, None]
        + eps_occ_right[None, None, :, None]
        - eps_vir_left[None, :, None, None]
        - eps_vir_right[None, None, None, :]
    )
    return np.sum(eri_ovov * eri_ovov / denominators)


def run_ump2(mol: gto.Mole, uhf_result: UHFResult) -> UMP2Result:
    occ_alpha = uhf_result.coefficients_alpha[:, : uhf_result.nalpha]
    occ_beta = uhf_result.coefficients_beta[:, : uhf_result.nbeta]
    vir_alpha = uhf_result.coefficients_alpha[:, uhf_result.nalpha :]
    vir_beta = uhf_result.coefficients_beta[:, uhf_result.nbeta :]

    eps_occ_alpha = uhf_result.orbital_energies_alpha[: uhf_result.nalpha]
    eps_occ_beta = uhf_result.orbital_energies_beta[: uhf_result.nbeta]
    eps_vir_alpha = uhf_result.orbital_energies_alpha[uhf_result.nalpha :]
    eps_vir_beta = uhf_result.orbital_energies_beta[uhf_result.nbeta :]

    ao_eri = mol.intor("int2e")
    eri_aa = transform_ao_eri_to_ovov(ao_eri, occ_alpha, vir_alpha, occ_alpha, vir_alpha)
    eri_ab = transform_ao_eri_to_ovov(ao_eri, occ_alpha, vir_alpha, occ_beta, vir_beta)
    eri_bb = transform_ao_eri_to_ovov(ao_eri, occ_beta, vir_beta, occ_beta, vir_beta)

    corr_aa = same_spin_mp2_energy(eri_aa, eps_occ_alpha, eps_vir_alpha)
    corr_ab = opposite_spin_mp2_energy(
        eri_ab,
        eps_occ_alpha,
        eps_occ_beta,
        eps_vir_alpha,
        eps_vir_beta,
    )
    corr_bb = same_spin_mp2_energy(eri_bb, eps_occ_beta, eps_vir_beta)
    total_corr = corr_aa + corr_ab + corr_bb

    return UMP2Result(
```

```
        uhf_energy=float(uhf_result.energy),
        correlation_energy_aa=float(corr_aa),
        correlation_energy_ab=float(corr_ab),
        correlation_energy_bb=float(corr_bb),
        ump2_correlation_energy=float(total_corr),
        total_energy=float(uhf_result.energy + total_corr),
        orbital_energies_alpha=uhf_result.orbital_energies_alpha.copy(),
        orbital_energies_beta=uhf_result.orbital_energies_beta.copy(),
        nocc_alpha=uhf_result.nalpha,
        nocc_beta=uhf_result.nbeta,
        nvir_alpha=vir_alpha.shape[1],
        nvir_beta=vir_beta.shape[1],
    )
```

UMP2 建立在 UHF 之上，所以它不再只有一套轨道，而是有：

- alpha 占据/虚轨道
- beta 占据/虚轨道

因此代码必须把相关能拆成三部分：

$$E_{\mathrm{UMP2}} = E_{aa} + E_{ab} + E_{bb}. \tag{12.6}$$

`same_spin_mp2_energy()` 为什么有 `0.25`

同自旋电子是不可分辨费米子，所以需要反对称化积分：

$$\langle ia||jb\rangle = \langle ia|jb\rangle - \langle ib|ja\rangle. \tag{12.7}$$

代码中的

```
antisymmetrized = eri_ovov - exchange_ovov
```

就是在做这件事。

`0.25` 的系数来自同自旋双激发求和中的重复计数修正。你暂时不必强行背住推导，但要知道这不是“拍脑袋写进去的常数”，而是来自指标对称性。

`opposite_spin_mp2_energy()` 为什么没有交换项

因为 alpha 和 beta 电子在自旋上已经可区分，所以 opposite-spin 项不需要像同自旋那样再减去交换重排项。程序里因此只保留

$$\sum \frac{(ia|jb)^2}{\Delta\varepsilon}. \tag{12.8}$$

读 `run_ump2()` 的顺序

最推荐的阅读顺序是：

1. 先看它如何从 `UHFResult` 里切出 alpha/beta 的占据与虚轨道。
2. 再看它如何分别构造 `eri_aa`、`eri_ab`、`eri_bb`。
3. 最后看它如何把三个相关能加总。

当你能说清楚这三步时，UMP2 的代码骨架其实已经完全掌握了。

继续学习资料

- PySCF MP2 用户文档：PySCF MP2 Guide
- Psi4NumPy 教学仓库，里面有 MP2 和 CC 相关参考实现：psi4/psi4numpy
- Crawford Lab 的量子化学编程项目主页，适合系统训练 HF/MP2/CC：Crawford Lab
- Frank Jensen 的《Introduction to Computational Chemistry》官方页面，适合把方法、基组和程序概念连起来：Wiley 页面

第十三章　DFT：RKS、UKS 与 `--xc` 泛函选择

本章目标

这一章要建立一个很重要的认识：在本项目里，DFT 不是“从零手写交换相关泛函”，而是学会如何把你已经懂的 SCF 框架，连接到 PySCF 提供的 KS-DFT 求解器上。

13.1　为什么 DFT 看起来像 HF，却又不是 HF

RKS/UKS 在程序外形上和 RHF/UHF 非常像：

$$\mathbf{FC} = \mathbf{SC}\varepsilon \tag{13.1}$$

的结构还在，仍然需要密度矩阵、重叠矩阵、迭代和收敛。

但最大的不同是，KS 矩阵中包含了交换相关势 v_{xc}。在真实工业级程序中，v_{xc} 的数值积分、网格、泛函导数都非常复杂，所以本项目把这部分交给 PySCF 处理。这是非常合理的教学取舍。

13.2　先看 `rks.py`：闭壳层 DFT 文件其实很整洁

```python
from __future__ import annotations

from dataclasses import dataclass

import numpy as np
from pyscf import dft, gto


@dataclass
class RKSResult:
    energy: float
    electronic_energy: float
    nuclear_repulsion: float
    orbital_energies: np.ndarray
    coefficients: np.ndarray
    density: np.ndarray
    iterations: int
    history: list[float]
```

```
    xc: str


def _validate_closed_shell_rks(mol: gto.Mole) -> None:
    if mol.spin != 0:
        raise ValueError(
            "This teaching example implements closed-shell RKS only. "
            "Please use spin = 0 or switch to UKS."
        )
    if mol.nelectron % 2 != 0:
        raise ValueError(
            "Closed-shell RKS requires an even number of electrons after charge is applied. "
        )


def run_rks(
    mol: gto.Mole,
    xc: str = "b3lyp",
    max_iter: int = 100,
    e_tol: float = 1.0e-10,
    d_tol: float = 1.0e-8,
    use_diis: bool = True,
    diis_space: int = 6,
) -> RKSResult:
    _validate_closed_shell_rks(mol)

    mf = dft.RKS(mol)
    mf.xc = xc
    mf.max_cycle = max_iter
    mf.conv_tol = e_tol
    mf.conv_tol_grad = max(d_tol, float(np.sqrt(e_tol)))
    mf.verbose = 0

    if use_diis:
        mf.diis_space = diis_space
    else:
        mf.DIIS = None
        mf.diis = None

    history: list[float] = []

    def callback(envs: dict[str, object]) -> None:
        energy = envs.get("e_tot")
        if energy is not None:
            history.append(float(energy))
```

```python

    mf.callback = callback
    total_energy = float(mf.kernel())
    if not mf.converged:
        raise RuntimeError("RKS did not converge within the iteration limit.")

    density = np.array(mf.make_rdm1(), copy=True)
    orbital_energies = np.array(mf.mo_energy, copy=True)
    coefficients = np.array(mf.mo_coeff, copy=True)
    e_nuc = float(mol.energy_nuc())
    e_elec = total_energy - e_nuc

    return RKSResult(
        energy=total_energy,
        electronic_energy=float(e_elec),
        nuclear_repulsion=e_nuc,
        orbital_energies=orbital_energies,
        coefficients=coefficients,
        density=density,
        iterations=len(history),
        history=history,
        xc=xc,
    )


def build_rks_reference_mf(mol: gto.Mole, rks_result: RKSResult) -> tuple[dft.RKS, np.ndarray]:
    mf = dft.RKS(mol)
    mf.xc = rks_result.xc
    nocc = mol.nelectron // 2
    mo_occ = np.zeros(mol.nao_nr())
    mo_occ[:nocc] = 2.0
    mf.mo_coeff = rks_result.coefficients
    mf.mo_energy = rks_result.orbital_energies
    mf.mo_occ = mo_occ
    mf.e_tot = rks_result.energy
    mf.converged = True
    return mf, mo_occ
```

RKSResult 和 RHFResult 很像

这是故意设计的。因为如果结果对象字段风格尽量统一，那么：

- CLI 更容易复用；
- 优化、频率模块更容易调用；

- 初学者也更容易对比“RHF 版本”和“RKS 版本”。

mf = dft.RKS(mol) 这一行意味着什么

它不是在“计算某个具体泛函”，而是在创建一个 **RKS 求解器对象**。真正选泛函的是后面的

```
mf.xc = xc
```

这就是命令行参数 `--xc` 最终落地的地方。

13.3 --xc 到底怎么理解

在我们的程序里，你可以写：

- `--xc lda,vwn`
- `--xc pbe`
- `--xc b3lyp`

初学者不需要一开始就记住所有泛函，只要先知道三件事：

1. `lda,vwn` 可以看成最基础的局域密度近似；
2. `pbe` 是很常见的 GGA；
3. `b3lyp` 是最常见的混合泛函之一。

项目本身并不解析这些名字的数学形式，而是把字符串交给 PySCF。也正因为如此，这个项目更适合作为“DFT 工作流入门”，而不是“XC 数值积分从零实现”教程。

13.4 回调函数 callback 为什么值得你学习

RKS/UKS 文件里有一段很适合初学者学 Python：

```python
history: list[float] = []

def callback(envs: dict[str, object]) -> None:
    energy = envs.get("e_tot")
    if energy is not None:
        history.append(float(energy))
```

这里体现了三个重要语法点：

- 列表 `history` 用来保存每一步能量；
- 内部函数 `callback` 会在 SCF 迭代过程中被反复调用；

- envs.get("e_tot") 是从字典里安全地取值。

所以这不是单纯的“技巧代码”，而是在告诉你：程序不仅能求最终结果，也可以记录整个收敛历史。

13.5 再看 uks.py：它和 UHF 的关系很直接

```python
from __future__ import annotations

from dataclasses import dataclass

import numpy as np
from pyscf import dft, gto


@dataclass
class UKSResult:
    energy: float
    electronic_energy: float
    nuclear_repulsion: float
    orbital_energies_alpha: np.ndarray
    orbital_energies_beta: np.ndarray
    coefficients_alpha: np.ndarray
    coefficients_beta: np.ndarray
    density_alpha: np.ndarray
    density_beta: np.ndarray
    iterations: int
    history: list[float]
    nalpha: int
    nbeta: int
    s2: float
    expected_s2: float
    spin_contamination: float
    xc: str


def run_uks(
    mol: gto.Mole,
    xc: str = "b3lyp",
    max_iter: int = 100,
    e_tol: float = 1.0e-10,
    d_tol: float = 1.0e-8,
    use_diis: bool = True,
    diis_space: int = 6,
) -> UKSResult:
```

```
    mf = dft.UKS(mol)
    mf.xc = xc
    mf.max_cycle = max_iter
    mf.conv_tol = e_tol
    mf.conv_tol_grad = max(d_tol, float(np.sqrt(e_tol)))
    mf.verbose = 0

    if use_diis:
        mf.diis_space = diis_space
    else:
        mf.DIIS = None
        mf.diis = None

    history: list[float] = []

    def callback(envs: dict[str, object]) -> None:
        energy = envs.get("e_tot")
        if energy is not None:
            history.append(float(energy))

    mf.callback = callback
    total_energy = float(mf.kernel())
    if not mf.converged:
        raise RuntimeError("UKS did not converge within the iteration limit.")

    density_alpha, density_beta = mf.make_rdm1()
    orbital_energies_alpha, orbital_energies_beta = mf.mo_energy
    coefficients_alpha, coefficients_beta = mf.mo_coeff
    nalpha, nbeta = mol.nelec
    s2, _ = mf.spin_square()
    expected_s2 = 0.5 * mol.spin * (0.5 * mol.spin + 1.0)
    e_nuc = float(mol.energy_nuc())
    e_elec = total_energy - e_nuc

    return UKSResult(
        energy=total_energy,
        electronic_energy=float(e_elec),
        nuclear_repulsion=e_nuc,
        orbital_energies_alpha=np.array(orbital_energies_alpha, copy=True),
        orbital_energies_beta=np.array(orbital_energies_beta, copy=True),
        coefficients_alpha=np.array(coefficients_alpha, copy=True),
        coefficients_beta=np.array(coefficients_beta, copy=True),
        density_alpha=np.array(density_alpha, copy=True),
        density_beta=np.array(density_beta, copy=True),
        iterations=len(history),
        history=history,
```

```
        nalpha=nalpha,
        nbeta=nbeta,
        s2=float(s2),
        expected_s2=float(expected_s2),
        spin_contamination=float(s2 - expected_s2),
        xc=xc,
    )


def build_uks_reference_mf(
    mol: gto.Mole,
    uks_result: UKSResult,
) -> tuple[dft.UKS, tuple[np.ndarray, np.ndarray]]:
    mf = dft.UKS(mol)
    mf.xc = uks_result.xc
    mo_occ_alpha = np.zeros(mol.nao_nr())
    mo_occ_beta = np.zeros(mol.nao_nr())
    mo_occ_alpha[: uks_result.nalpha] = 1.0
    mo_occ_beta[: uks_result.nbeta] = 1.0
    mf.mo_coeff = (uks_result.coefficients_alpha, uks_result.coefficients_beta)
    mf.mo_energy = (uks_result.orbital_energies_alpha, uks_result.orbital_energies_beta)
    mf.mo_occ = (mo_occ_alpha, mo_occ_beta)
    mf.e_tot = uks_result.energy
    mf.converged = True
    return mf, (mo_occ_alpha, mo_occ_beta)
```

UKS 和 UHF 的关系，很像 RKS 和 RHF 的关系。也就是说：

- 如果体系闭壳层，RKS 往往最自然；
- 如果体系开壳层，UKS 往往更合适；
- 如果你关心自旋污染，UKS 也可以像 UHF 那样计算 $\langle S^2 \rangle$。

为什么 UKSResult 里还保留 s2

因为开壳层 DFT 虽然不是 UHF，但仍然可能出现类似的自旋污染问题。所以项目里把

- `s2`
- `expected_s2`
- `spin_contamination`

都保存下来，这能让你在比较 UHF 和 UKS 时看到自旋行为是否改善。

继续学习资料

- PySCF DFT 用户文档：PySCF DFT Guide
- PySCF 总体用户指南，方便你继续查看 SCF、DFT、CC 等模块：PySCF User Guide
- PySCF 的快速上手页面，适合把课堂公式快速变成可运行代码：PySCF Quickstart
- Psi4Education 项目介绍，适合跟着教学实验理解基组、HF、MP2、CCSD 等方法差异：Psi4Education

第十四章　CCSD：把单参考相关方法再推进一步

本章目标

这一章不是要你立刻手推完整的 CCSD 振幅方程，而是要让你明白：为什么项目在 CCSD 部分选择“自己准备 RHF 参考，再调用 PySCF 的 CC 求解器”，以及 `t1/t2` 这些振幅在程序里是什么地位。

14.1　CCSD 和 MP2 的关系

MP2 是二阶微扰修正；CCSD 则把单激发和双激发振幅放进指数波函数

$$|\Psi_{\mathrm{CC}}\rangle = e^{\hat{T}}|\Phi_0\rangle, \quad \hat{T} = \hat{T}_1 + \hat{T}_2. \tag{14.1}$$

你可以把它粗略理解成：MP2 是“修正一次能量”，而 CCSD 是“通过非线性振幅方程系统地重求相关波函数”。它更贵，但通常也更可靠。

14.2　完整看一遍 `ccsd.py`

```python
from __future__ import annotations

from dataclasses import dataclass

import numpy as np
from pyscf import cc, gto, scf

from .rhf import RHFResult


@dataclass
class CCSDResult:
    rhf_energy: float
    ccsd_correlation_energy: float
    total_energy: float
    orbital_energies: np.ndarray
    nocc: int
    nvir: int
```

```
    t1: np.ndarray
    t2: np.ndarray
    t1_norm: float
    t2_norm: float
    converged: bool


def build_reference_mf(mol: gto.Mole, rhf_result: RHFResult) -> tuple[scf.RHF, np.ndarray
    ]:
    mf = scf.RHF(mol)
    nocc = mol.nelectron // 2
    mo_occ = np.zeros(mol.nao_nr())
    mo_occ[:nocc] = 2.0
    mf.mo_coeff = rhf_result.coefficients
    mf.mo_energy = rhf_result.orbital_energies
    mf.mo_occ = mo_occ
    mf.e_tot = rhf_result.energy
    mf.converged = True
    return mf, mo_occ


def run_ccsd(mol: gto.Mole, rhf_result: RHFResult) -> CCSDResult:
    mf, mo_occ = build_reference_mf(mol, rhf_result)
    ccsd_solver = cc.CCSD(mf, mo_coeff=rhf_result.coefficients, mo_occ=mo_occ)
    ccsd_solver.verbose = 0
    ccsd_solver.conv_tol = 1.0e-10
    correlation_energy, t1, t2 = ccsd_solver.kernel()

    nocc = mol.nelectron // 2
    nvir = rhf_result.coefficients.shape[1] - nocc
    total_energy = rhf_result.energy + correlation_energy

    return CCSDResult(
        rhf_energy=float(rhf_result.energy),
        ccsd_correlation_energy=float(correlation_energy),
        total_energy=float(total_energy),
        orbital_energies=rhf_result.orbital_energies.copy(),
        nocc=nocc,
        nvir=nvir,
        t1=t1.copy(),
        t2=t2.copy(),
        t1_norm=float(np.linalg.norm(t1)),
        t2_norm=float(np.linalg.norm(t2)),
        converged=bool(ccsd_solver.converged),
    )
```

为什么先要 build_reference_mf

这一步和我们在优化、频率模块里做的事情是同一个套路：我们已经有了自己写出来的 RHF 结果，但 PySCF 的 CCSD 求解器需要一个“像样的 mean-field 对象”作为参考态输入。

所以这里不是重新做一遍 HF，而是把已有的

- mo_coeff
- mo_energy
- mo_occ
- e_tot

重新塞回一个 scf.RHF 对象里。

correlation_energy, t1, t2 = ccsd_solver.kernel() 的含义

这一行非常关键：

- correlation_energy 是 CCSD 相对 RHF 的相关修正；
- t1 是单激发振幅；
- t2 是双激发振幅。

程序随后又计算了

- t1_norm
- t2_norm

它们不是最终物理量，但非常适合作为“结果量级检查”。如果某次计算里 t1、t2 的范数异常大，你就应该警惕参考态是否不好，或者体系是否超出了单参考方法的适用范围。

学习上的现实建议

如果你现在刚学完 RHF/UHF，不建议立刻去手推完整 CCSD 方程然后硬写。更好的路线是：先理解 CCSD 的对象关系和程序接口，再去读 Psi4NumPy 或 Crawford 项目里的 CC 参考实现。

继续学习资料

- PySCF CC 用户文档：PySCF CC Guide
- Psi4NumPy 仓库中的 Coupled-Cluster 教学实现：psi4/psi4numpy
- Psi4 教程入口，后续如果你想把 Python 调用和量化输入格式联系起来会很有帮助：Psi4 Tutorials
- Crawford Lab 主页，上面维护着经典量化编程训练项目：Crawford Lab

第十五章　几何优化与扫描：从单点能走向结构探索

本章目标

这一章是项目功能跨度最大的一章。你会看到程序如何从“固定几何求一个能量”，成长为“自动挪动原子、寻找更低能结构、沿着键长/键角/二面角扫描势能面”的工具。

15.1　几何优化本质上在做什么

几何优化的目标不是直接最小化波函数参数，而是最小化**核坐标**上的总能量：

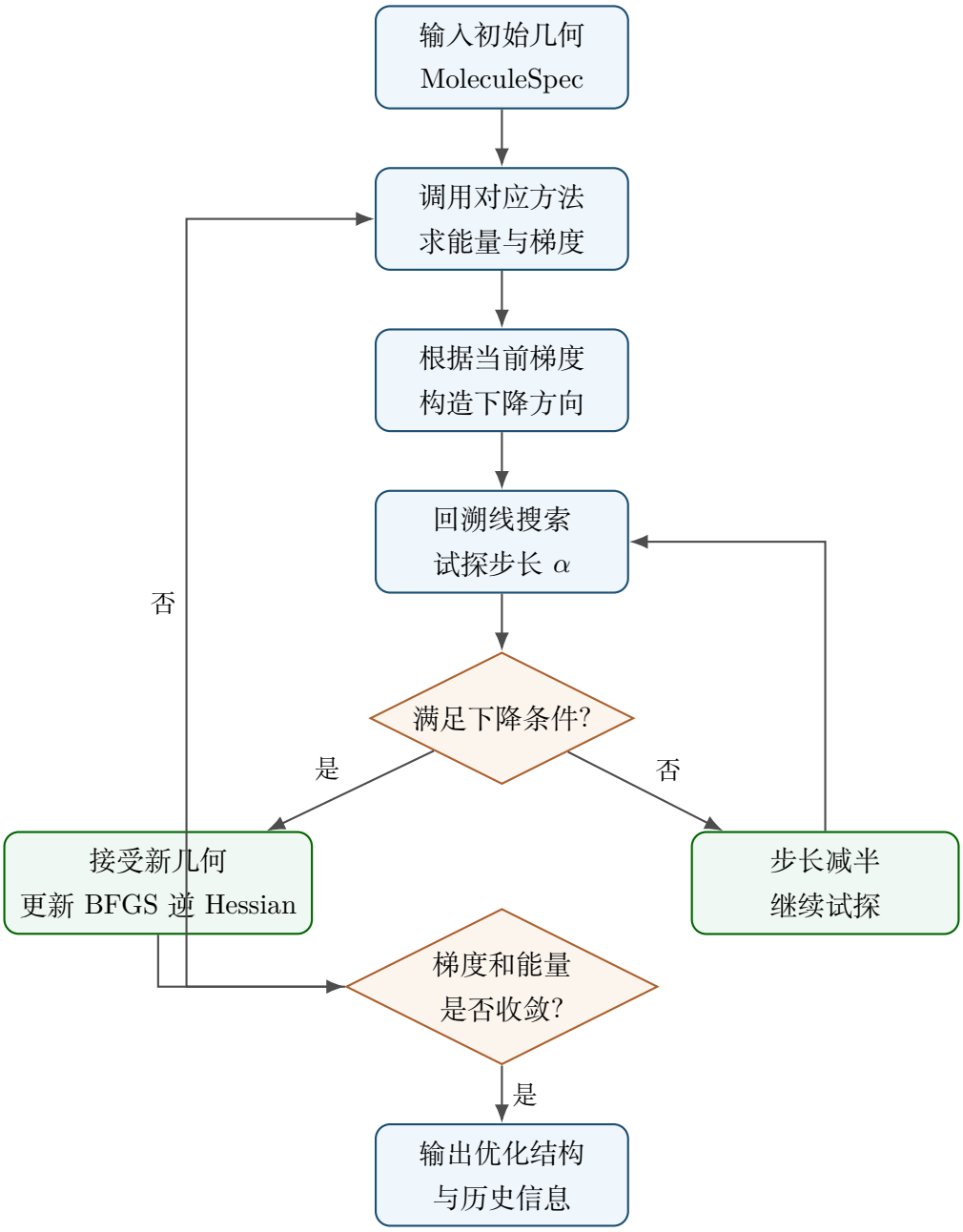
$$E = E(\mathbf{R}). \tag{15.1}$$

因此你需要的不只是单点能量，还需要梯度：

$$\mathbf{g} = \frac{\partial E}{\partial \mathbf{R}}. \tag{15.2}$$

项目里的优化模块选择了一个很适合教学的路线：

1. 先调用 RHF/UHF/RKS/UKS 求单点；
2. 再调用 PySCF 的梯度求解器；
3. 用一个简化版 BFGS 思路更新几何；
4. 用回溯线搜索确保步长不要太激进。



这个流程图对应的就是 `optimize_geometry()` 的主循环：每一轮都在“算梯度、提方向、试步长、检查收敛”之间来回。

15.2 先看 `optimize.py` 的文件地图

- `OptimizationStep`：记录每一步的能量、最大梯度、RMS 梯度和步长。
- `OptimizationResult`：记录最终优化结果。
- `evaluate_energy_and_gradient()`：统一封装“给定几何，算能量和梯度”。
- `make_spec_with_bohr_coords()` / `format_spec_in_original_unit()`：负责几何字符串和单位转换。

- optimize_geometry()：真正执行优化循环。

```
@dataclass
class OptimizationStep:
    step: int
    energy: float
    max_gradient: float
    rms_gradient: float
    step_length: float


@dataclass
class OptimizationResult:
    method: str
    xc: str | None
    initial_energy: float
    final_energy: float
    converged: bool
    iterations: int
    optimized_spec: MoleculeSpec
    final_gradient: np.ndarray
    history: list[OptimizationStep]
    final_result: RHFResult | UHFResult | RKSResult | UKSResult


PenaltyFunction = Callable[[np.ndarray], tuple[float, np.ndarray]]


def evaluate_energy_and_gradient(
    spec: MoleculeSpec,
    method: str,
    xc: str,
    max_iter: int,
    e_tol: float,
    d_tol: float,
    use_diis: bool,
    diis_space: int,
    penalty_function: PenaltyFunction | None = None,
) -> tuple[float, np.ndarray, RHFResult | UHFResult | RKSResult | UKSResult]:
    mol = build_molecule(spec)
    if method == "rhf":
        result = run_rhf(
            mol,
            max_iter=max_iter,
            e_tol=e_tol,
            d_tol=d_tol,
```

```
            use_diis=use_diis,
            diis_space=diis_space,
        )
        mf, _ = build_rhf_reference_mf(mol, result)
    elif method == "rks":
        result = run_rks(
            mol,
            xc=xc,
            max_iter=max_iter,
            e_tol=e_tol,
            d_tol=d_tol,
            use_diis=use_diis,
            diis_space=diis_space,
        )
        mf, _ = build_rks_reference_mf(mol, result)
    elif method == "uhf":
        result = run_uhf(
            mol,
            max_iter=max_iter,
            e_tol=e_tol,
            d_tol=d_tol,
            use_diis=use_diis,
            diis_space=diis_space,
        )
        mf, _ = build_uhf_reference_mf(mol, result)
    elif method == "uks":
        result = run_uks(
            mol,
            xc=xc,
            max_iter=max_iter,
            e_tol=e_tol,
            d_tol=d_tol,
            use_diis=use_diis,
            diis_space=diis_space,
        )
        mf, _ = build_uks_reference_mf(mol, result)
    else:
        raise ValueError(f"Unsupported optimization method '{method}'.")

    grad_solver = mf.nuc_grad_method()
    grad_solver.verbose = 0
    gradient = grad_solver.kernel()
    total_energy = float(result.energy)
    total_gradient = np.array(gradient, copy=True)

    if penalty_function is not None:
```

```
        _, coords_bohr = parse_atom_string(spec.atom)
        penalty_energy, penalty_gradient = penalty_function(coords_bohr)
        total_energy += float(penalty_energy)
        total_gradient = total_gradient + np.array(penalty_gradient, copy=False)

    return total_energy, total_gradient, result
```

为什么要有 evaluate_energy_and_gradient()

因为优化器并不想关心你当前是 RHF、UHF、RKS 还是 UKS。它只想知道：

> 给我一个几何，我要得到当前能量和梯度。

所以这个函数本质上是一个“统一接口层”。这是一种非常值得初学者学习的软件设计思路。

15.3 optimize_geometry() 的主循环该怎么读

```
def optimize_geometry(
    spec: MoleculeSpec,
    method: str,
    xc: str = "b3lyp",
    max_opt_steps: int = 30,
    grad_tol: float = 1.0e-4,
    energy_tol: float = 1.0e-8,
    max_step_size: float = 0.2,
    max_iter: int = 100,
    scf_e_tol: float = 1.0e-10,
    scf_d_tol: float = 1.0e-8,
    use_diis: bool = True,
    diis_space: int = 6,
    penalty_function: PenaltyFunction | None = None,
) -> OptimizationResult:
    if method not in {"rhf", "uhf", "rks", "uks"}:
        raise ValueError("Geometry optimization currently supports RHF, UHF, RKS, and UKS."
            )

    symbols, coords_input = parse_atom_string(spec.atom)
    coords_bohr = convert_coords(coords_input, spec.unit, "Bohr")
    current_spec = make_spec_with_bohr_coords(symbols, coords_bohr, spec)
    current_energy, current_gradient, current_result = evaluate_energy_and_gradient(
        current_spec,
        method,
        xc,
        max_iter,
        scf_e_tol,
```

```
        scf_d_tol,
        use_diis,
        diis_space,
        penalty_function,
    )

    initial_energy = current_energy
    history: list[OptimizationStep] = []
    dimension = current_gradient.size
    inverse_hessian = np.eye(dimension)
    previous_energy = None
    converged = False

    for step in range(1, max_opt_steps + 1):
        gradient_flat = current_gradient.reshape(-1)
        max_gradient = float(np.max(np.abs(gradient_flat)))
        rms_gradient = float(np.sqrt(np.mean(gradient_flat**2)))

        if previous_energy is not None and abs(current_energy - previous_energy) <
            energy_tol and max_gradient < grad_tol:
            converged = True
            break

        if previous_energy is None and max_gradient < grad_tol:
            history.append(
                OptimizationStep(
                    step=step,
                    energy=current_energy,
                    max_gradient=max_gradient,
                    rms_gradient=rms_gradient,
                    step_length=0.0,
                )
            )
            converged = True
            break

        direction = -inverse_hessian @ gradient_flat
        if float(np.dot(direction, gradient_flat)) >= 0.0:
            direction = -gradient_flat.copy()
            inverse_hessian = np.eye(dimension)

        direction_norm = float(np.linalg.norm(direction))
        if direction_norm > max_step_size:
            direction *= max_step_size / direction_norm

        trial_alpha = 1.0
```

```
        accepted = False
        c1 = 1.0e-4
        best_trial: tuple[
            float,
            np.ndarray,
            RHFResult | UHFResult | RKSResult | UKSResult,
            np.ndarray,
        ] | None = None

        for _ in range(12):
            trial_coords_bohr = coords_bohr + trial_alpha * direction.reshape(-1, 3)
            trial_spec = make_spec_with_bohr_coords(symbols, trial_coords_bohr, spec)
            try:
                trial_energy, trial_gradient, trial_result = evaluate_energy_and_gradient(
                    trial_spec,
                    method,
                    xc,
                    max_iter,
                    scf_e_tol,
                    scf_d_tol,
                    use_diis,
                    diis_space,
                    penalty_function,
                )
            except RuntimeError:
                trial_alpha *= 0.5
                continue
            if trial_energy <= current_energy + c1 * trial_alpha * float(np.dot(
                gradient_flat, direction)):
                best_trial = (trial_energy, trial_gradient, trial_result, trial_coords_bohr)
                accepted = True
                break
            trial_alpha *= 0.5

        if not accepted or best_trial is None:
            history.append(
                OptimizationStep(
                    step=step,
                    energy=current_energy,
                    max_gradient=max_gradient,
                    rms_gradient=rms_gradient,
                    step_length=0.0,
                )
            )
            break

```

```
        trial_energy, trial_gradient, trial_result, trial_coords_bohr = best_trial
        step_vector = (trial_coords_bohr - coords_bohr).reshape(-1)
        new_gradient_flat = trial_gradient.reshape(-1)
        y_vector = new_gradient_flat - gradient_flat
        ys = float(np.dot(y_vector, step_vector))

        if ys > 1.0e-10:
            identity = np.eye(dimension)
            rho = 1.0 / ys
            outer_sy = np.outer(step_vector, y_vector)
            outer_ys = np.outer(y_vector, step_vector)
            inverse_hessian = (
                (identity - rho * outer_sy)
                @ inverse_hessian
                @ (identity - rho * outer_ys)
                + rho * np.outer(step_vector, step_vector)
            )
        else:
            inverse_hessian = np.eye(dimension)

        history.append(
            OptimizationStep(
                step=step,
                energy=trial_energy,
                max_gradient=float(np.max(np.abs(new_gradient_flat))),
                rms_gradient=float(np.sqrt(np.mean(new_gradient_flat**2))),
                step_length=float(np.linalg.norm(step_vector)),
            )
        )

        previous_energy = current_energy
        coords_bohr = trial_coords_bohr
        current_spec = make_spec_with_bohr_coords(symbols, coords_bohr, spec)
        current_energy = trial_energy
        current_gradient = trial_gradient
        current_result = trial_result

    optimized_spec = format_spec_in_original_unit(current_spec, spec.unit)
    return OptimizationResult(
        method=method,
        xc=xc if method in {"rks", "uks"} else None,
        initial_energy=initial_energy,
        final_energy=current_energy,
        converged=converged,
        iterations=len(history),
        optimized_spec=optimized_spec,
```

```
        final_gradient=np.array(current_gradient, copy=True),
        history=history,
        final_result=current_result,
    )
```

第一层：所有坐标先转成 Bohr

量子化学内部计算常常统一在原子单位下进行。所以代码先把输入几何转换成 Bohr，再在最后按用户输入单位转回去。这能避免优化过程中各种长度单位混杂。

第二层：逆 Hessian 近似矩阵在干什么

```
inverse_hessian = np.eye(dimension)
```

这一行意味着：优化一开始先用单位矩阵当作 Hessian 逆矩阵的初猜。之后程序根据新旧梯度和位移向量做 BFGS 更新：

$$\mathbf{H}_{k+1}^{-1} = (\mathbf{I} - \rho \mathbf{s}\mathbf{y}^T)\mathbf{H}_k^{-1}(\mathbf{I} - \rho \mathbf{y}\mathbf{s}^T) + \rho \mathbf{s}\mathbf{s}^T. \tag{15.3}$$

这里你不必强行背公式，但至少要知道：

- `step_vector` 对应 $\mathbf{s}$；
- `y_vector` 对应梯度差 $\mathbf{y}$；
- 这一步是在不断改进“下一步往哪里走”的方向判断。

第三层：为什么还要回溯线搜索

因为哪怕方向是下降方向，步子太大也可能把结构带到很糟糕的位置。于是代码通过

- 先试 `trial_alpha = 1.0`
- 如果不满足下降条件就乘 `0.5`

逐步缩小步长。这是非常经典、也很适合教学实现的策略。

优化里最容易忽略的事

优化收敛并不等于结构一定是“真极小值”。它只说明梯度足够小。要判断它是不是稳定极小值，下一步通常要做频率分析，看是否存在虚频。

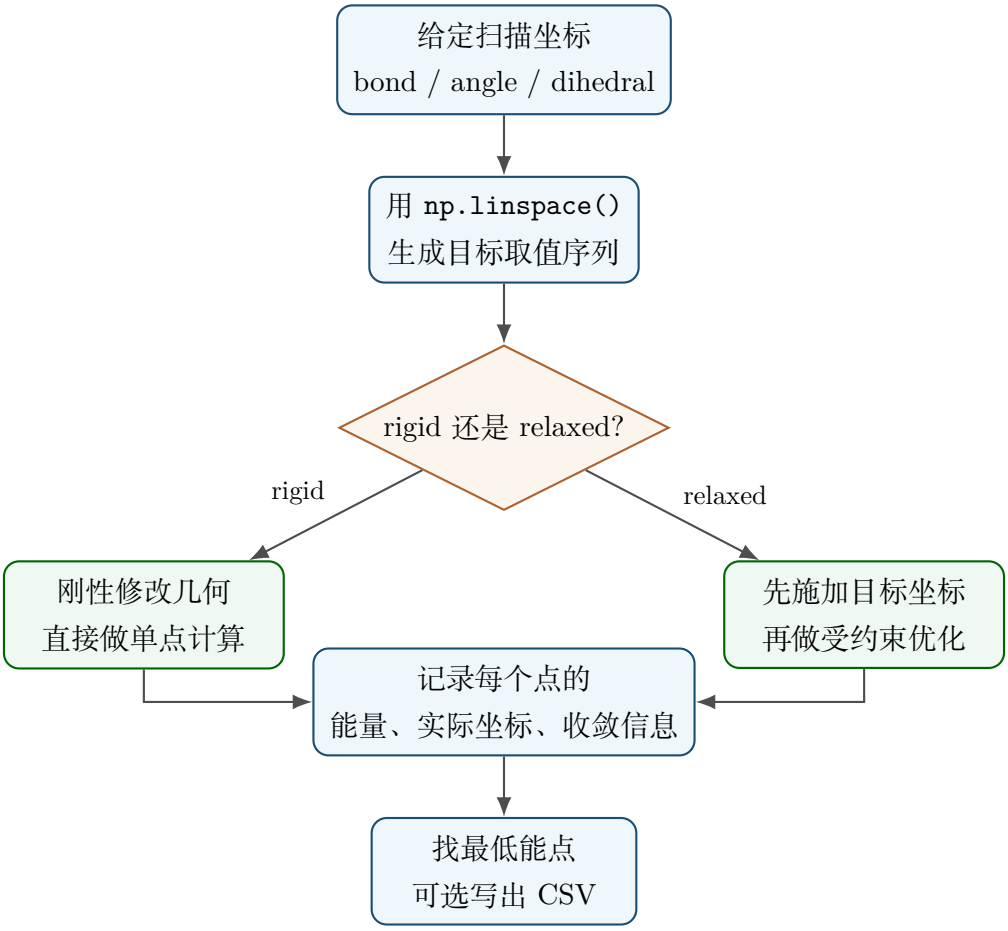
15.4 扫描：为什么要同时支持 rigid 和 relaxed

扫描的目标是沿某个内部坐标采样势能面。例如：

- 改变 O–H 键长，看能量怎么变；
- 改变 H–O–H 键角，看哪一个角度能量最低；
- 改变四个原子定义的二面角，看构象差异。

但“扫描”有两种完全不同的做法：

1. rigid scan：只强行改这个坐标，其余几何不放松；
2. relaxed scan：把这个坐标约束在目标值附近，其余坐标重新优化。



把它和 `scan.py` 对起来看时，你会很容易发现：两种扫描方式最大的差异，不在“坐标定义”，而在“每个扫描点内部到底是做一次单点，还是做一次优化”。

15.5 `scan.py` 文件虽然长，但可以分成四层

- 坐标辅助函数：识别 bond/angle/dihedral 的指标数、单位、当前值。
- 约束势函数：把“我要固定某个坐标”变成惩罚能。
- 单点接口：`evaluate_single_point()` 统一分发到 RHF/UHF/RKS/UKS，以及 MP2/UMP2/CCSD。
- 两类扫描主循环：`rigid_scan()` 与 `relaxed_scan()`。

```python
def coordinate_arity(coordinate_type: str) -> int:
    return {"bond": 2, "angle": 3, "dihedral": 4}[coordinate_type]


def coordinate_display_unit(coordinate_type: str, spec_unit: str) -> str:
    if coordinate_type == "bond":
        return spec_unit
    return "deg"


def coordinate_value_display(coords: np.ndarray, coordinate_type: str, atoms: tuple[int, ...]) -> float:
    if coordinate_type == "bond":
        return bond_length(coords, atoms) # type: ignore[arg-type]
    if coordinate_type == "angle":
        return angle_degrees(coords, atoms) # type: ignore[arg-type]
    if coordinate_type == "dihedral":
        return dihedral_degrees(coords, atoms) # type: ignore[arg-type]
    raise ValueError(f"Unsupported coordinate type '{coordinate_type}'.")


def coordinate_value_internal(coords_bohr: np.ndarray, coordinate_type: str, atoms: tuple[int, ...]) -> float:
    if coordinate_type == "bond":
        return bond_length(coords_bohr, atoms) # type: ignore[arg-type]
    if coordinate_type == "angle":
        return angle_radians(coords_bohr, atoms) # type: ignore[arg-type]
    if coordinate_type == "dihedral":
        return dihedral_radians(coords_bohr, atoms) # type: ignore[arg-type]
    raise ValueError(f"Unsupported coordinate type '{coordinate_type}'.")


def set_coordinate_rigid(
    coords: np.ndarray,
    coordinate_type: str,
    atoms: tuple[int, ...],
    target_value: float,
) -> np.ndarray:
    if coordinate_type == "bond":
        return set_bond_length_rigid(coords, atoms, target_value) # type: ignore[arg-type]
    if coordinate_type == "angle":
        return set_angle_rigid(coords, atoms, target_value) # type: ignore[arg-type]
    if coordinate_type == "dihedral":
        return set_dihedral_rigid(coords, atoms, target_value) # type: ignore[arg-type]
    raise ValueError(f"Unsupported coordinate type '{coordinate_type}'.")
```

```python


def target_value_to_internal(target_value: float, coordinate_type: str, spec_unit: str) -> float:
    if coordinate_type == "bond":
        return convert_length_value(target_value, spec_unit, "Bohr")
    if coordinate_type in {"angle", "dihedral"}:
        return float(np.deg2rad(target_value))
    raise ValueError(f"Unsupported coordinate type '{coordinate_type}'.")


def displacement_unit_for_coordinate(coordinate_type: str) -> str:
    if coordinate_type == "bond":
        return "length"
    return "angular"


def periodic_difference(value: float, target: float, coordinate_type: str) -> float:
    delta = value - target
    if coordinate_type == "dihedral":
        return wrap_angle_radians(delta)
    return delta


def numerical_gradient(
    energy_fn: Callable[[np.ndarray], float],
    coords: np.ndarray,
    step: float = 1.0e-5,
) -> np.ndarray:
    gradient = np.zeros_like(coords)
    for atom_index in range(coords.shape[0]):
        for axis in range(3):
            plus = np.array(coords, copy=True)
            minus = np.array(coords, copy=True)
            plus[atom_index, axis] += step
            minus[atom_index, axis] -= step
            gradient[atom_index, axis] = (energy_fn(plus) - energy_fn(minus)) / (2.0 * step)
    return gradient


def build_coordinate_penalty(
    coordinate_type: str,
    atoms: tuple[int, ...],
    target_internal: float,
    penalty_k: float,
```

```
) -> Callable[[np.ndarray], tuple[float, np.ndarray]]:
    def penalty_energy(coords_bohr: np.ndarray) -> float:
        current_internal = coordinate_value_internal(coords_bohr, coordinate_type, atoms)
        delta = periodic_difference(current_internal, target_internal, coordinate_type)
        return 0.5 * penalty_k * delta * delta

    def penalty(coords_bohr: np.ndarray) -> tuple[float, np.ndarray]:
        energy = float(penalty_energy(coords_bohr))
        gradient = numerical_gradient(penalty_energy, coords_bohr)
        return energy, gradient

    return penalty
```

惩罚函数为什么很重要

relaxed scan 没有直接“硬锁死”某个坐标，而是用

$$E_{\text{penalty}} = \frac{1}{2}k(q - q_0)^2 \tag{15.4}$$

给目标坐标一个谐振子型惩罚。这样做的好处是：

- 其余自由度还能自然放松；
- 对角度和二面角扫描比较稳；
- 和几何优化模块的接口容易衔接。

15.6 读 rigid scan 和 relaxed scan 的主循环

```
def evaluate_single_point(
    spec: MoleculeSpec,
    method: str,
    xc: str,
    max_iter: int,
    e_tol: float,
    d_tol: float,
    use_diis: bool,
    diis_space: int,
) -> tuple[float, SinglePointResult]:
    mol = build_molecule(spec)
    if method == "rhf":
        result = run_rhf(mol, max_iter=max_iter, e_tol=e_tol, d_tol=d_tol, use_diis=
            use_diis, diis_space=diis_space)
        return result.energy, result
    if method == "rks":
        result = run_rks(
```

```
            mol,
            xc=xc,
            max_iter=max_iter,
            e_tol=e_tol,
            d_tol=d_tol,
            use_diis=use_diis,
            diis_space=diis_space,
        )
        return result.energy, result
    if method == "mp2":
        rhf_result = run_rhf(mol, max_iter=max_iter, e_tol=e_tol, d_tol=d_tol, use_diis=use_diis, diis_space=diis_space)
        result = run_mp2(mol, rhf_result)
        return result.total_energy, result
    if method == "ccsd":
        rhf_result = run_rhf(mol, max_iter=max_iter, e_tol=e_tol, d_tol=d_tol, use_diis=use_diis, diis_space=diis_space)
        result = run_ccsd(mol, rhf_result)
        return result.total_energy, result
    if method == "uhf":
        result = run_uhf(mol, max_iter=max_iter, e_tol=e_tol, d_tol=d_tol, use_diis=use_diis, diis_space=diis_space)
        return result.energy, result
    if method == "uks":
        result = run_uks(
            mol,
            xc=xc,
            max_iter=max_iter,
            e_tol=e_tol,
            d_tol=d_tol,
            use_diis=use_diis,
            diis_space=diis_space,
        )
        return result.energy, result
    if method == "ump2":
        uhf_result = run_uhf(mol, max_iter=max_iter, e_tol=e_tol, d_tol=d_tol, use_diis=use_diis, diis_space=diis_space)
        result = run_ump2(mol, uhf_result)
        return result.total_energy, result
    raise ValueError(f"Unsupported scan method '{method}'.")


def rigid_scan(
    spec: MoleculeSpec,
    method: str,
    xc: str,
```

```
    coordinate_type: str,
    atoms: tuple[int, ...],
    start_value: float,
    stop_value: float,
    num_points: int,
    max_iter: int,
    e_tol: float,
    d_tol: float,
    use_diis: bool,
    diis_space: int,
) -> ScanResult:
    symbols, coords = parse_atom_string(spec.atom)
    values = np.linspace(start_value, stop_value, num_points)
    points: list[ScanPoint] = []

    for index, target_value in enumerate(values, start=1):
        scan_coords = set_coordinate_rigid(coords, coordinate_type, atoms, float(
            target_value))
        scan_spec = MoleculeSpec(
            atom=format_atom_string(symbols, scan_coords),
            basis=spec.basis,
            charge=spec.charge,
            spin=spec.spin,
            unit=spec.unit,
            title=spec.title,
        )
        actual = coordinate_value_display(scan_coords, coordinate_type, atoms)
        try:
            energy, result = evaluate_single_point(
                scan_spec,
                method,
                xc,
                max_iter,
                e_tol,
                d_tol,
                use_diis,
                diis_space,
            )
            converged = True
        except RuntimeError:
            energy = float("inf")
            result = None
            converged = False
        points.append(
            ScanPoint(
                index=index,
```

```
            target_value=float(target_value),
            actual_value=actual,
            energy=float(energy),
            converged=converged,
            spec=scan_spec,
            result=result,
        )
    )

    best_point = min(points, key=lambda point: point.energy)
    return ScanResult(
        mode="rigid",
        coordinate_type=coordinate_type,
        method=method,
        xc=xc if method in {"rks", "uks"} else None,
        atoms=atoms,
        value_unit=coordinate_display_unit(coordinate_type, spec.unit),
        points=points,
        best_point=best_point,
    )
```

```
def relaxed_scan(
    spec: MoleculeSpec,
    method: str,
    xc: str,
    coordinate_type: str,
    atoms: tuple[int, ...],
    start_value: float,
    stop_value: float,
    num_points: int,
    penalty_k: float,
    max_opt_steps: int,
    grad_tol: float,
    opt_energy_tol: float,
    max_step_size: float,
    max_iter: int,
    scf_e_tol: float,
    scf_d_tol: float,
    use_diis: bool,
    diis_space: int,
) -> ScanResult:
    if method not in {"rhf", "uhf", "rks", "uks"}:
        raise ValueError("Relaxed scans currently support RHF, UHF, RKS, and UKS.")

    symbols, coords_input = parse_atom_string(spec.atom)
    coords_bohr = convert_coords(coords_input, spec.unit, "Bohr")
```

```
    display_values = np.linspace(start_value, stop_value, num_points)
    points: list[ScanPoint] = []
    working_coords_bohr = np.array(coords_bohr, copy=True)

    for index, target_display in enumerate(display_values, start=1):
        if coordinate_type == "bond":
            target_for_rigid = convert_length_value(float(target_display), spec.unit, "Bohr
                ")
        else:
            target_for_rigid = float(target_display)

        start_coords_bohr = set_coordinate_rigid(
            working_coords_bohr,
            coordinate_type,
            atoms,
            target_for_rigid,
        )
        start_spec = MoleculeSpec(
            atom=format_atom_string(symbols, start_coords_bohr),
            basis=spec.basis,
            charge=spec.charge,
            spin=spec.spin,
            unit="Bohr",
            title=spec.title,
        )
        target_internal = target_value_to_internal(float(target_display), coordinate_type,
            spec.unit)
        try:
            opt_result = optimize_geometry(
                start_spec,
                method=method,
                xc=xc,
                max_opt_steps=max_opt_steps,
                grad_tol=grad_tol,
                energy_tol=opt_energy_tol,
                max_step_size=max_step_size,
                max_iter=max_iter,
                scf_e_tol=scf_e_tol,
                scf_d_tol=scf_d_tol,
                use_diis=use_diis,
                diis_space=diis_space,
                penalty_function=build_coordinate_penalty(coordinate_type, atoms,
                    target_internal, penalty_k),
            )

            optimized_symbols, optimized_coords_out = parse_atom_string(opt_result.
```

```
                optimized_spec.atom)
            optimized_coords_bohr = convert_coords(optimized_coords_out, opt_result.
                optimized_spec.unit, "Bohr")
            translation_bohr = anchor_translation_bohr(
                coordinate_type,
                atoms,
                coords_bohr,
                optimized_coords_bohr,
            )
            optimized_coords_bohr = optimized_coords_bohr + translation_bohr
            optimized_coords_out = convert_coords(optimized_coords_bohr, "Bohr", spec.unit)
            working_coords_bohr = optimized_coords_bohr
            actual_display = coordinate_value_display(optimized_coords_out, coordinate_type
                , atoms)
            true_energy = float(opt_result.final_result.energy)
            point_spec = MoleculeSpec(
                atom=format_atom_string(optimized_symbols, optimized_coords_out),
                basis=spec.basis,
                charge=spec.charge,
                spin=spec.spin,
                unit=spec.unit,
                title=spec.title,
            )
            point_result = opt_result.final_result
            point_converged = opt_result.converged
        except RuntimeError:
            fallback_coords_out = convert_coords(start_coords_bohr, "Bohr", spec.unit)
            actual_display = coordinate_value_display(fallback_coords_out, coordinate_type,
                 atoms)
            true_energy = float("inf")
            point_spec = MoleculeSpec(
                atom=format_atom_string(symbols, fallback_coords_out),
                basis=spec.basis,
                charge=spec.charge,
                spin=spec.spin,
                unit=spec.unit,
                title=spec.title,
            )
            point_result = None
            point_converged = False
            opt_result = None

        points.append(
            ScanPoint(
                index=index,
                target_value=float(target_display),
```

```
            actual_value=actual_display,
            energy=true_energy,
            converged=point_converged,
            spec=point_spec,
            result=point_result,
            optimization_result=opt_result,
        )
    )

    best_point = min(points, key=lambda point: point.energy)
    return ScanResult(
        mode="relaxed",
        coordinate_type=coordinate_type,
        method=method,
        xc=xc if method in {"rks", "uks"} else None,
        atoms=atoms,
        value_unit=coordinate_display_unit(coordinate_type, spec.unit),
        points=points,
        best_point=best_point,
    )


def write_scan_csv(scan_result: ScanResult, path: str) -> None:
    output_path = Path(path)
    output_path.parent.mkdir(parents=True, exist_ok=True)
    with output_path.open("w", encoding="utf-8", newline="") as handle:
        writer = csv.writer(handle)
        writer.writerow(
            [
                "index",
                "coordinate_type",
                "target_value",
                "actual_value",
                "value_unit",
                "energy_hartree",
                "converged",
            ]
        )
        for point in scan_result.points:
            writer.writerow(
                [
                    point.index,
                    scan_result.coordinate_type,
                    f"{point.target_value:.10f}",
                    f"{point.actual_value:.10f}",
                    scan_result.value_unit,
```

```
                f"{point.energy:.12f}",
                point.converged,
            ]
        )
```

rigid scan 在做什么

它的逻辑非常直接：

1. 用 `np.linspace()` 生成一串目标坐标值；
2. 每个目标值都构造一个新几何；
3. 对每个几何做一次单点；
4. 把所有点打包成 `ScanResult`；
5. 最后找出最低能点。

relaxed scan 为什么更长

因为它每个扫描点内部都还要再做一次几何优化。它的思路是：

1. 先把当前工作几何刚性调整到目标值附近；
2. 再调用 `optimize_geometry()` 做受约束优化；
3. 得到优化后的结构后，继续作为下一个扫描点的起点。

这会让扫描更平滑，也更接近真实势能面最小能路径。

为什么还要写 `write_scan_csv()`

因为扫描结果天然适合画图。CSV 可以让你后续直接用：

- Excel
- Origin
- Python 的 `matplotlib`

去画能量随键长、键角或二面角变化的曲线。

继续学习资料

- PySCF 几何优化用户文档：PySCF Geometry Optimization
- PySCF 的 `geomopt` 接口文档：PySCF geomopt Interface
- geomeTRIC 约束优化官方文档，理解 relaxed scan 里的“柔性约束”很有帮助：geomeTRIC

Constraints

- PySCF 的通用使用说明，其中 scanner 概念对理解结构优化流程很有帮助：PySCF Using Guide

第十六章　频率分析：检查结构是极小值还是鞍点

本章目标

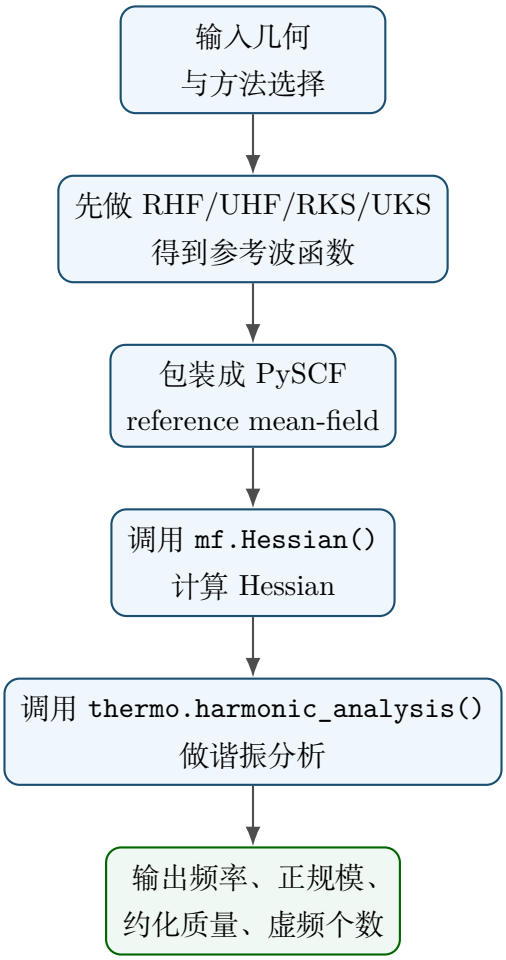
频率分析看似是“优化之后的附加功能”，实际上它承担着非常关键的判断任务：你找到的结构到底是稳定极小值，还是过渡态，还是根本没有优化好。

16.1　频率分析为什么离不开 Hessian

频率分析的核心对象是 Hessian：

$$H_{A\alpha,B\beta} = \frac{\partial^2 E}{\partial R_{A\alpha}\partial R_{B\beta}}. \tag{16.1}$$

也就是总能量对核坐标的二阶导数矩阵。把质量加权后的 Hessian 对角化，就能得到振动频率和正规模。



频率分析这部分的流程之所以清晰，是因为它几乎完全复用了前面已经建立好的单点求解接口。真正新增的核心对象，其实只有 Hessian 和谐振分析结果。

16.2 完整看一遍 frequency.py

```
from __future__ import annotations

from dataclasses import dataclass

import numpy as np
from pyscf.hessian import thermo

from .geometry import MoleculeSpec
from .rks import RKSResult, build_rks_reference_mf, run_rks
from .rhf import RHFResult, build_molecule, build_rhf_reference_mf, run_rhf
from .uks import UKSResult, build_uks_reference_mf, run_uks
from .uhf import UHFResult, build_uhf_reference_mf, run_uhf


ReferenceResult = RHFResult | UHFResult | RKSResult | UKSResult

```

```python

@dataclass
class FrequencyResult:
    method: str
    xc: str | None
    energy: float
    hessian: np.ndarray
    frequencies_cm1: np.ndarray
    frequencies_au: np.ndarray
    normal_modes: np.ndarray
    reduced_masses: np.ndarray
    force_constants_au: np.ndarray
    force_constants_dyne: np.ndarray
    vib_temperatures: np.ndarray
    num_imaginary: int
    freq_error: int
    reference_result: ReferenceResult


def run_frequency(
    spec: MoleculeSpec,
    method: str,
    xc: str = "b3lyp",
    max_iter: int = 100,
    e_tol: float = 1.0e-10,
    d_tol: float = 1.0e-8,
    use_diis: bool = True,
    diis_space: int = 6,
) -> FrequencyResult:
    if method not in {"rhf", "uhf", "rks", "uks"}:
        raise ValueError("Frequency analysis currently supports RHF, UHF, RKS, and UKS.")

    mol = build_molecule(spec)

    if method == "rhf":
        result = run_rhf(
            mol,
            max_iter=max_iter,
            e_tol=e_tol,
            d_tol=d_tol,
            use_diis=use_diis,
            diis_space=diis_space,
        )
        mf, _ = build_rhf_reference_mf(mol, result)
    elif method == "uhf":
        result = run_uhf(
```

```
            mol,
            max_iter=max_iter,
            e_tol=e_tol,
            d_tol=d_tol,
            use_diis=use_diis,
            diis_space=diis_space,
        )
        mf, _ = build_uhf_reference_mf(mol, result)
    elif method == "rks":
        result = run_rks(
            mol,
            xc=xc,
            max_iter=max_iter,
            e_tol=e_tol,
            d_tol=d_tol,
            use_diis=use_diis,
            diis_space=diis_space,
        )
        mf, _ = build_rks_reference_mf(mol, result)
    else:
        result = run_uks(
            mol,
            xc=xc,
            max_iter=max_iter,
            e_tol=e_tol,
            d_tol=d_tol,
            use_diis=use_diis,
            diis_space=diis_space,
        )
        mf, _ = build_uks_reference_mf(mol, result)

    hessian_solver = mf.Hessian()
    hessian_solver.verbose = 0
    hessian = np.array(hessian_solver.kernel(), copy=True)
    harmonic_data = thermo.harmonic_analysis(mol, hessian, imaginary_freq=True)

    frequencies_cm1 = np.array(harmonic_data["freq_wavenumber"], copy=True)
    num_imaginary = int(np.count_nonzero(np.abs(frequencies_cm1.imag) > 1.0e-8))

    return FrequencyResult(
        method=method,
        xc=xc if method in {"rks", "uks"} else None,
        energy=float(result.energy),
        hessian=hessian,
        frequencies_cm1=frequencies_cm1,
        frequencies_au=np.array(harmonic_data["freq_au"], copy=True),
```

```
        normal_modes=np.array(harmonic_data["norm_mode"], copy=True),
        reduced_masses=np.array(harmonic_data["reduced_mass"], copy=True),
        force_constants_au=np.array(harmonic_data["force_const_au"], copy=True),
        force_constants_dyne=np.array(harmonic_data["force_const_dyne"], copy=True),
        vib_temperatures=np.array(harmonic_data["vib_temperature"], copy=True),
        num_imaginary=num_imaginary,
        freq_error=int(harmonic_data["freq_error"]),
        reference_result=result,
    )
```

文件的骨架和优化模块很像

你会发现它的套路和优化模块非常接近：

1. 先根据 `method` 选择 RHF/UHF/RKS/UKS；
2. 再把结果包装成 PySCF reference mean-field；
3. 然后调用 `mf.Hessian()`；
4. 最后再把 Hessian 交给 `thermo.harmonic_analysis()`。

这正说明了一个很重要的软件结构思想：

> 只要前面的“结果对象”和“reference_mf 构造器”设计得统一，后面的高级功能就能很顺地接上。

`num_imaginary` 是怎么来的

程序里用了

```
num_imaginary = int(np.count_nonzero(np.abs(frequencies_cm1.imag) > 1.0e-8))
```

这表示如果某个频率有明显虚部，就把它计成一个虚频。

对结构判断来说：

- 0 个虚频，通常表示极小值；
- 1 个虚频，常常表示一阶鞍点，也就是过渡态候选；
- 多个虚频，通常说明结构还不稳定，或者方法/初猜有问题。

为什么结果里还保存正规模、约化质量、力常数

因为频率分析不只是“给一串 cm^{-1}”数字。很多后续分析都依赖这些量：

- `normal_modes`：看每个振动模式到底是哪几个原子在动；

- `reduced_masses`：理解模式质量特征；
- `force_constants`：理解势能面的局部刚性。

实践提醒

如果你在未优化的结构上直接做频率分析，看到虚频并不奇怪。频率分析最好放在优化之后再做，而且要明确你想找的是极小值还是过渡态。

继续学习资料

- PySCF Hessian/API 文档，里面直接给出了 `harmonic_analysis()` 的接口：PySCF Hessian API
- PySCF 几何优化文档，因为频率分析通常和优化配套使用：PySCF Geometry Optimization
- PySCF 快速上手页，适合继续查找梯度、Hessian、溶剂、DFT 等示例：PySCF Quickstart
- Jensen 的教材官方页，适合把频率、热化学和势能面概念系统补齐：Wiley 页面

第十七章　学完这一轮之后，下一步该怎么继续

阶段性路线图

如果你已经能跟着这份讲义把项目读到这里，你就已经不是“只会跑黑箱程序”的状态了。下一阶段最值得继续补的，不是盲目堆方法名，而是按一条有层次的路线往前走。

17.1　最推荐的继续路线

1. 先把本项目里所有结果对象和 CLI 路线彻底读熟。
2. 自己重写一个最小版 MP2，再验证与当前项目结果是否一致。
3. 继续补可视化，例如把扫描结果自动画成图。
4. 学解析梯度与数值梯度的区别，理解为什么优化会更稳定。
5. 如果对电子相关特别感兴趣，再继续读 CCSD(T)、CASSCF、NEVPT2。

17.2　哪些资料最适合从“会跑”走向“会写”

通用学习资料

- PySCF 用户指南总入口：PySCF User Guide
- Psi4NumPy：psi4/psi4numpy
- Psi4Education：Psi4Education
- Crawford Lab：Crawford Lab
- Frank Jensen 教材官方页：Wiley 页面

第十八章　本册的最后总结

如果你走到这里，真正需要带走的已经不只是某一个公式，而是一整套“量子化学小程序是怎样长出来的”认识：

1. Hartree 引入了平均场思想。
2. Hartree–Fock 通过 Slater 行列式修复了费米子反对称性。
3. RHF 在程序中体现为：矩阵、密度、Fock、对角化、自洽循环。
4. DIIS 不是魔法，只是利用最近若干步的信息来稳住 SCF。
5. UHF 的本质，是把单一闭壳层图景扩展成自旋分辨图景。
6. MP2/UMP2 让你第一次真正看到电子相关在程序里的张量结构。
7. RKS/UKS 让你看到 HF 风格代码怎样过渡到 DFT 工作流。
8. CCSD 说明更高层相关方法往往建立在统一 reference wavefunction 接口上。
9. 优化、扫描和频率分析告诉你：量子化学程序并不只会算一个单点能。
10. 真正的量子化学编程，不是死背公式，而是把公式拆成可靠的数据结构、统一接口与数值步骤。

如果你已经能把 `geometry.py`、`rhf.py`、`uhf.py` 这一层读懂，并且开始理解 `mp2.py`、`rks.py`、`optimize.py`、`scan.py`、`frequency.py` 这些模块，那么你就已经走到了“可以自己继续扩项目”的门口。

附录：本册建议实际运行的命令

```
cd simple_hf_water
python3 -m pip install -r requirements.txt

# 最基础的 RHF 水分子计算
python3 rhf_sto3g_water.py --method rhf

# 显示 RHF 迭代历史
python3 rhf_sto3g_water.py --method rhf --show-history

# 关闭 DIIS, 看收敛行为变化
python3 rhf_sto3g_water.py --method rhf --no-diis

# 开壳层 OH 自由基 UHF
python3 rhf_sto3g_water.py --method uhf --xyz examples/oh_radical.xyz --spin 1

# 在 RHF 结果基础上做 MP2
python3 rhf_sto3g_water.py --method mp2

# 用 B3LYP 做闭壳层 RKS
python3 rhf_sto3g_water.py --method rks --xc b3lyp

# 对 OH 自由基做 UKS
python3 rhf_sto3g_water.py --method uks --xc pbe --xyz examples/oh_radical.xyz --spin 1

# 计算 CCSD 能量
python3 rhf_sto3g_water.py --method ccsd

# RHF 几何优化
python3 rhf_sto3g_water.py --method rhf --optimize

# RKS 几何优化
python3 rhf_sto3g_water.py --method rks --xc b3lyp --optimize

# 刚性键角扫描
python3 rhf_sto3g_water.py --method rhf --scan rigid \
  --scan-coordinate angle --scan-atoms 2,1,3 \
  --scan-start 90 --scan-stop 120 --scan-points 7
```

```

# 柔性键角扫描并导出 CSV
python3 rhf_sto3g_water.py --method rks --xc b3lyp --scan relaxed \
  --scan-coordinate angle --scan-atoms 2,1,3 \
  --scan-start 95 --scan-stop 115 --scan-points 5 \
  --scan-output results/angle_scan.csv

# 键长扫描
python3 rhf_sto3g_water.py --method rhf --scan rigid \
  --scan-coordinate bond --scan-atoms 1,2 \
  --scan-start 0.8 --scan-stop 1.3 --scan-points 8

# 二面角扫描
python3 rhf_sto3g_water.py --method uhf --scan rigid \
  --scan-coordinate dihedral --scan-atoms 1,2,3,4 \
  --scan-start -180 --scan-stop 180 --scan-points 13

# 频率分析
python3 rhf_sto3g_water.py --method rhf --frequency
```

建议你不要只是跑命令，而是把每个输出字段和对应源码变量逐一对上。只有这样，代码和理论才会真正连起来。